# Foundations of Computing & Programming

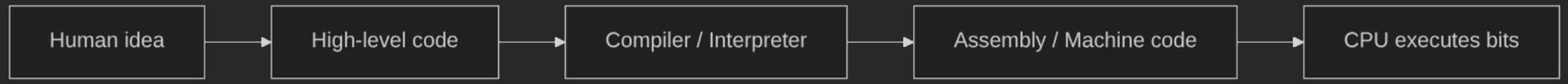
Python Course · Module 1

ElnurBDa

# What is Programming?

Programming is the process of giving instructions to a computer in a language it understands.

Computers themselves only understand **machine code**, so programming languages allow humans to write instructions more easily.




Think of it as translation layers that convert human steps into electrical signals.

Programming is used for:

- Automation
- Web development
- **Cybersecurity**
- Data analysis
- System administration

Real-world examples:

- ATMs
- Browsers
- Mobile apps
- Network scanners (Nmap, etc.)

---

# Why Programming Matters

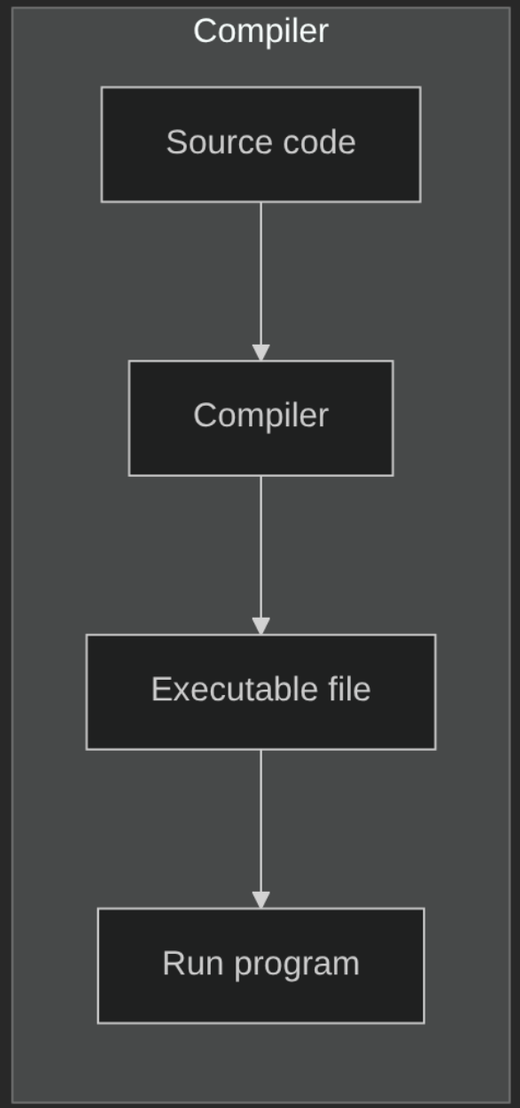
Programming allows you to:

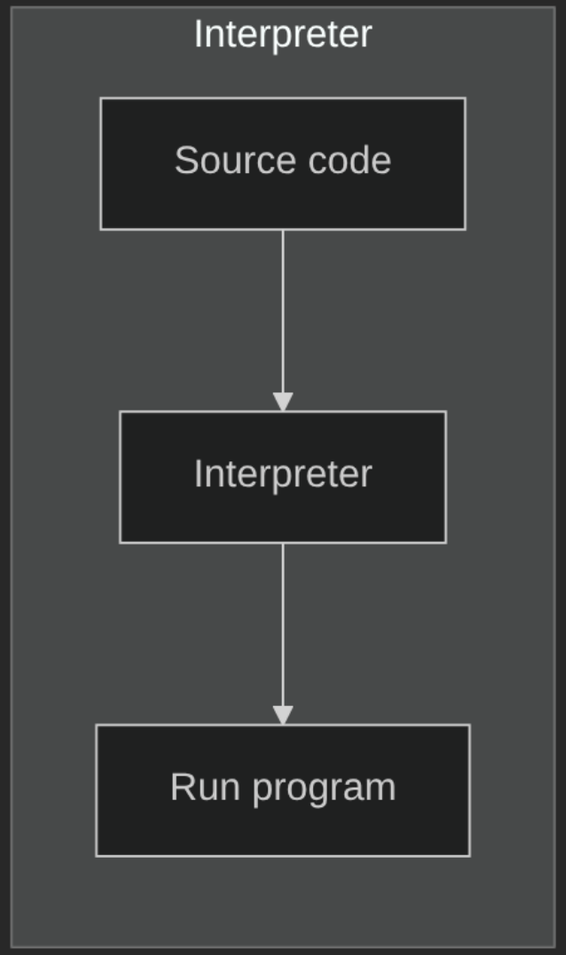
- Solve problems
- Automate repetitive tasks
- Build software
- Understand how computers actually work
- Strengthen cybersecurity skills (scripting, automation)

It is a foundational skill for modern IT and cybersecurity professionals.

---

# Compilers vs Interpreters



# Compiler

A **compiler** converts the entire source code into machine code **before** the program runs.

Characteristics:

- Produces a standalone executable
- Faster execution
- Errors found before running

Examples:

- C
- C++
- Go
- Rust

▒▒▒ Example:

hello.c

```c
#include <stdio.h>

int main() {
    printf("Hello, world!\n");
    return 0;
}
```

```bash
# compile
gcc hello.c -o hello
# run
chmod +x hello
./hello
# Result: Hello, world!
```

# Interpreter

An **interpreter** executes code **line by line**.

Characteristics:

- More flexible
- Easier to test small parts
- Slower execution
- Errors show up while running

Examples:

- Python
- JavaScript
- Ruby

▒▒▒ Example:

hello.py

```python
print("Hello, world!")
```

```python
# run
python hello.py
# Result: Hello, world!
```

# Algorithms – Introduction

An **algorithm** is a step-by-step procedure to solve a problem.

Characteristics of good algorithms:

- Clarity
- Efficiency
- Consistency
- Finite steps

Common examples in daily life:

- Sorting items
- Making tea
- Following a recipe
- Unlocking your phone

---

# Algorithm Building Blocks

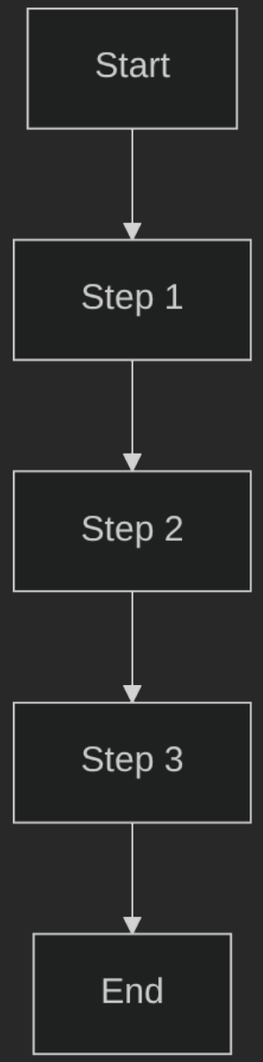
All algorithms are built from a few fundamental building blocks:

1. **Sequence** — steps executed in order
2. **Selection** — making decisions (if/else)
3. **Iteration** — repeating steps (loops)
4. **Variables** — storing data
5. **Input/Output** — getting and displaying data

These building blocks can be combined to solve any computational problem.

---

# Sequence



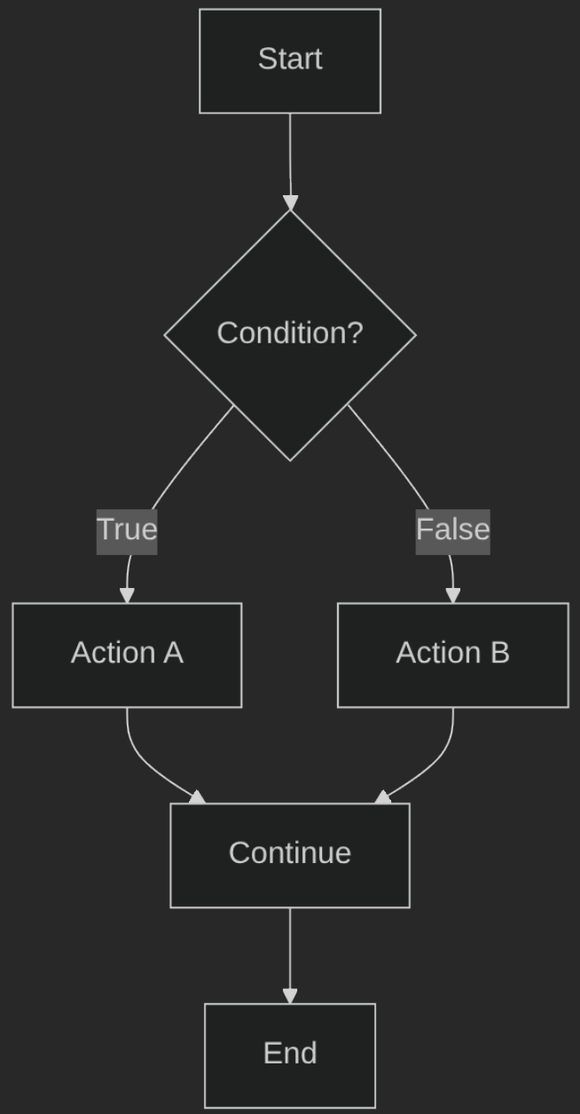

**Sequence** means executing steps one after another, in order.

Example (Making coffee):

1. Grind coffee beans
2. Boil water
3. Pour water over coffee
4. Serve

Each step must complete before the next begins.

# Selection (Conditionals)



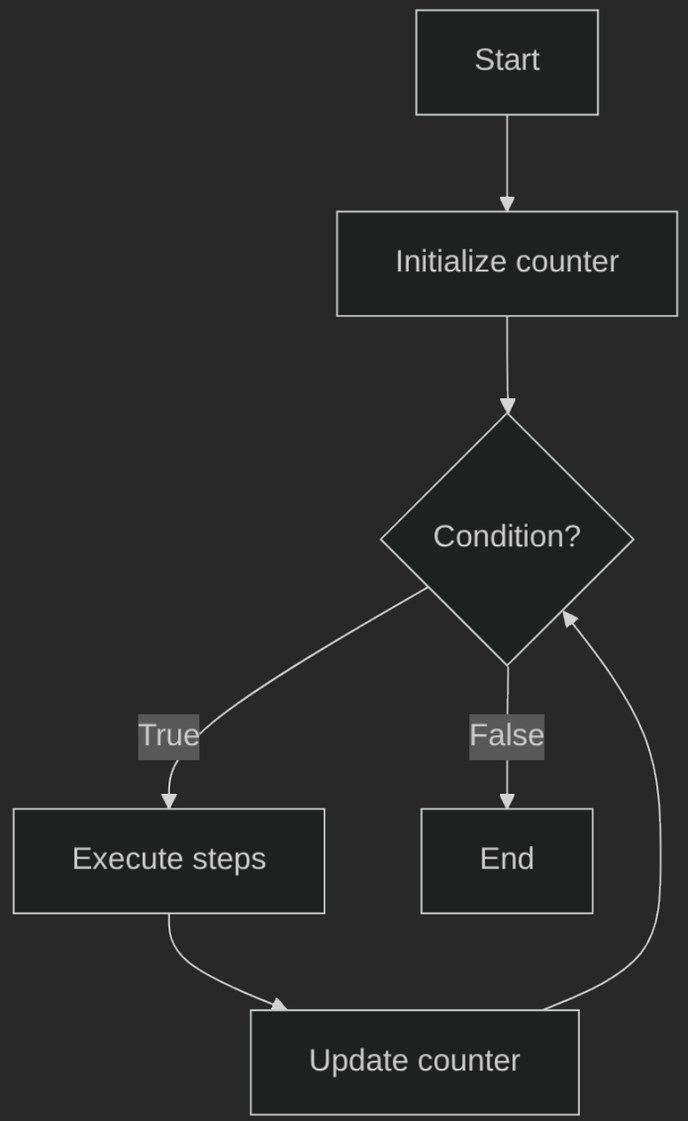

**Selection** allows algorithms to make decisions based on conditions.

Example:

```
if temperature > 30:
    print("It's hot!")
else:
    print("It's cool!")
```

The algorithm chooses different paths based on the condition.

# Iteration (Loops)



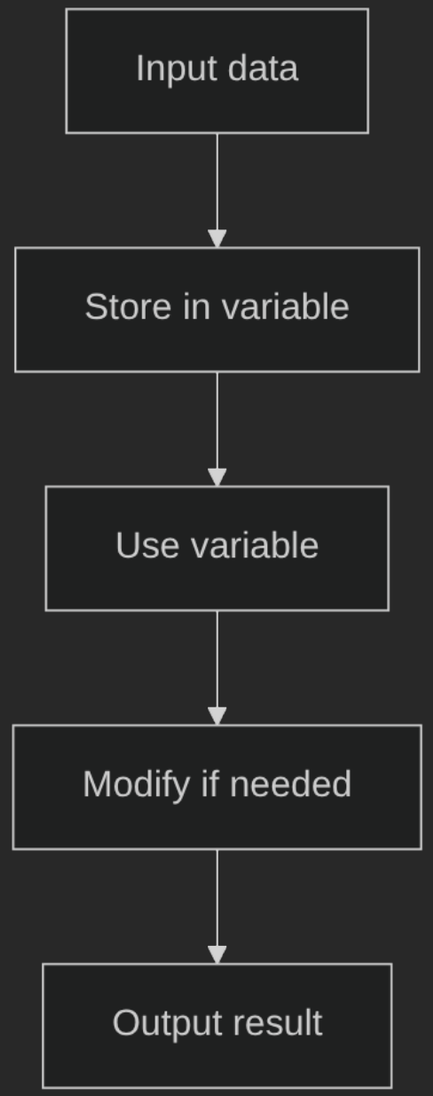

**Iteration** repeats a set of steps until a condition is met.

Example (Count to 5):

```
count = 1
while count <= 5:
    print(count)
    count = count + 1
```

Loops help automate repetitive tasks efficiently.

# Variables and Data Storage



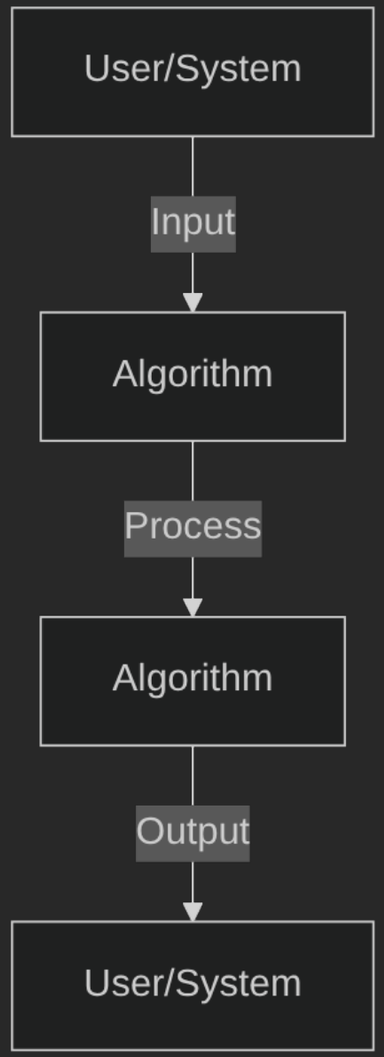

**Variables** store data that can be used and modified during algorithm execution.

Example:

```
name = "Alice"
age = 25
message = "Hello, " + name
print(message)
```

Variables act as containers that hold values for later use.

# Input and Output



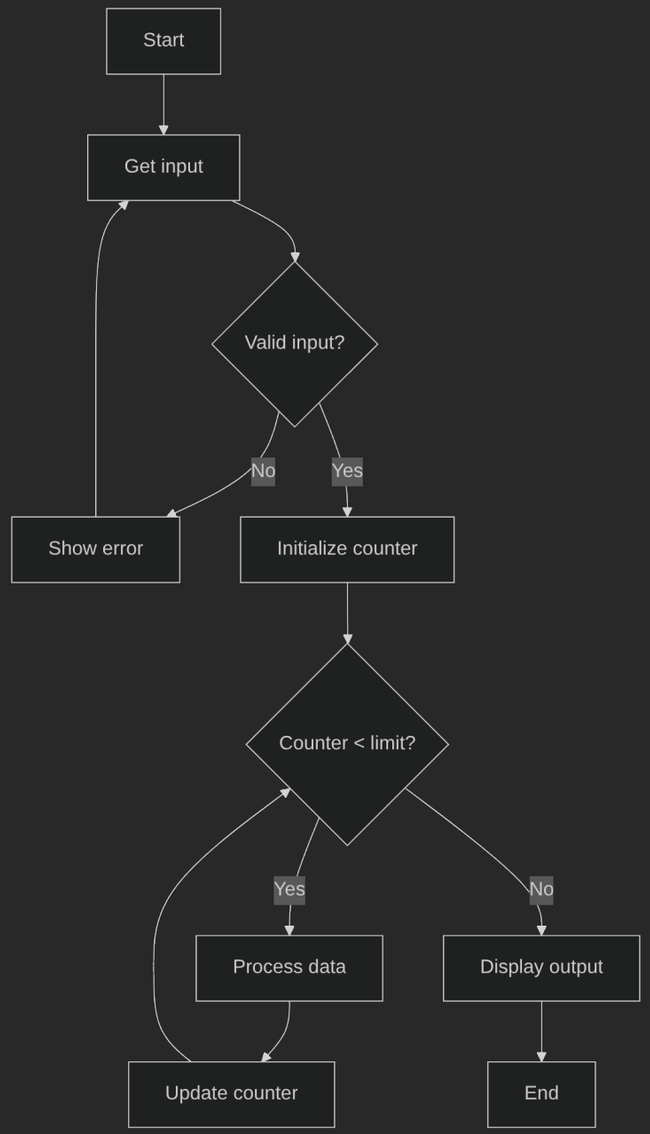

**Input** gets data from the user or system. **Output** displays or returns results.

Example:

```
name = input("Enter your name: ")
print("Hello, " + name)
```

Input provides data to work with; output shows the results.

# Combining Building Blocks



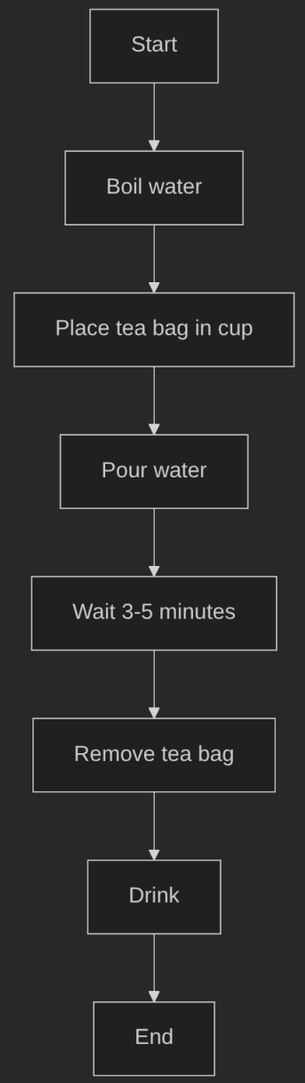

Real algorithms combine multiple building blocks:

This combines:

- **Sequence**: steps in order
- **Selection**: validation check
- **Iteration**: processing loop
- **Variables**: storing counter and data
- **Input/Output**: getting data and displaying results

# Algorithm Examples

**Making Tea** (Sequential Algorithm):




Real-life algorithm example (Making Tea):

1. Boil water
2. Place tea bag in cup
3. Pour water
4. Wait 3–5 minutes
5. Remove tea bag
6. Drink

# Create a Simple Algorithm — Activity

Write an algorithm for:

1. Logging into a website
2. Buying a product online
3. Starting a computer

Goal: describe steps **clearly** so that even a computer could follow them.

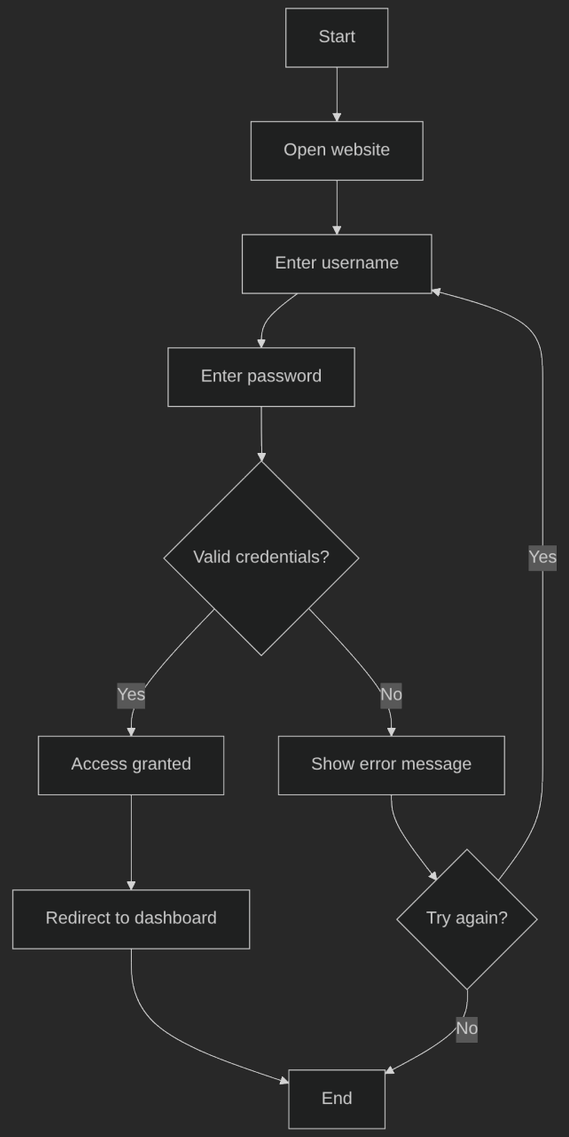
---

# Logging into a Website – Solution

**Logging into a Website** (Algorithm with Selection and Iteration):

This algorithm demonstrates:

- **Sequence**: steps executed in order
- **Selection**: checking if credentials are valid
- **Iteration**: option to try again if login fails



# Algorithm Design Checklist

| Step | Questions to ask |
|---|---|
| Inputs | What data do I need? What format? |
| Outputs | What should be produced? |
| Steps | In what order? Any loops or decisions? |
| Edge cases | What if input is missing/invalid? |
| Representation | Pseudocode, flowchart, or plain text? |

Comparison:

- **Pseudocode** — quick, language-agnostic
- **Flowchart** — great for visual branching
- **Plain text** — fast brainstorming, less structure

---

# Installing Python

Check whether Python is installed:

```
python3 --version
```

Install on Linux (Debian/Ubuntu):

```
sudo apt update
sudo apt install python3
```

Install on macOS (Homebrew):

```
brew install python
```

Install on Windows:

- Use Microsoft Store
- Or download installer from python.org

Enter interactive mode (REPL):

```
python3
```

Exit REPL with:

```
exit()
# or press Ctrl+D on Linux/macOS
```

---

# Running Python Code

Run a script file:

```
python3 script.py
```

Simple script example (hello.py):

```
print("Hello, world!")
```

Then run:

```
python3 hello.py
```

---

# Virtual Environments

Later modules will install packages like requests, Flask, etc.

Without virtual environments:

- Different projects may **fight over versions**
- You may not remember which scripts require which packages

With virtual environments:

- Each project can have its **own dependencies**
- You can safely experiment without breaking other projects

We start using this pattern now so it feels natural later.

---

# Creating and Activating a venv

Inside projects/module1_hello/:

```
python3 -m venv .venv
```

Activate it:

```
source .venv/bin/activate   # Linux/macOS
# or on Windows (PowerShell):
# .venv\Scripts\Activate.ps1
```

Your prompt should now show something like:

```
(.venv) user@machine:~/.../module1_hello$
```

To deactivate later:

```
deactivate
```

---

# Installing a Test Package

While still inside the (.venv):

```
pip install --upgrade pip
pip install requests
pip freeze
```

You should see requests and its dependencies listed.

Optional: save them to requirements.txt:

```
pip freeze > requirements.txt
```

Now this project knows exactly **which versions** it uses.

---

# Thanks!

By ElnurBDa

# Python Basics

Python Course · Module 2

ElnurBDa

# Variables

A variable stores a **reference** to a value.

Created by assignment:

```
name = "Alice"
age = 20
height = 1.75
```

Rules:

- Start with a letter or _
- Letters, digits, underscores allowed
- Case-sensitive: count ≠ Count

---

# Data Types

### int

Whole numbers, positive or negative.
Example:

```
age = 25
```

### float

Decimal numbers.

```
pi = 3.14
```

### str

Text data in quotes.

```
city = "Baku"
```

### bool

Logical values: True or False.

```
is_student = True
```

### NoneType

Represents “no value”.

```
result = None
```

---

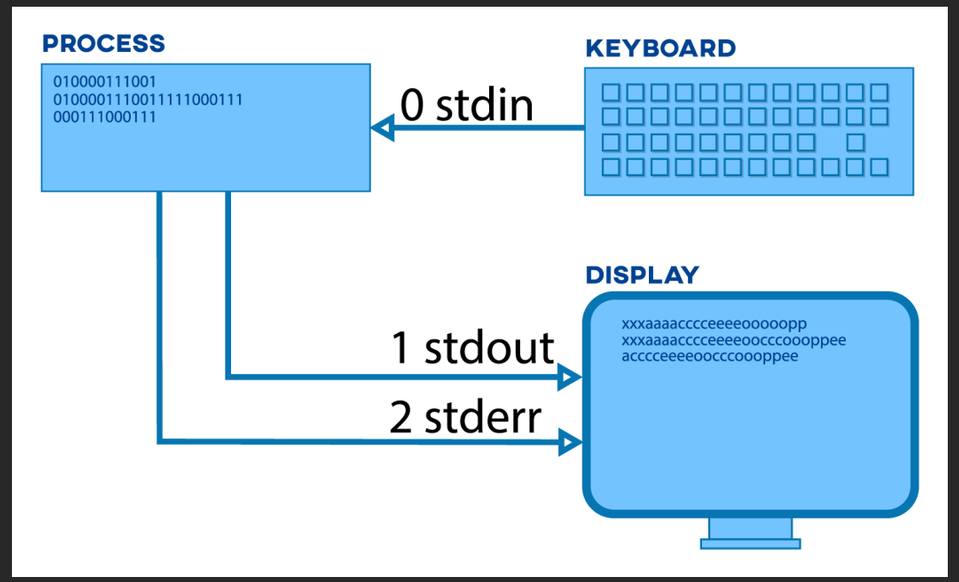
# Input & Output

Output (stdout)

```
name = "ElnurBDa"
print("Hello")
print("User:", name)
print(f"The F String is used by {name}")
```

[finished]

```
Hello
User: ElnurBDa
The F String is used by ElnurBDa
```




Input (stdin)

```
name = input("Enter your name: ")
```

`input()` **always** returns `str`.

Convert for numeric use:

```
age = int(input("Enter age: "))
```

# Arithmetic Operators

## Operator Summary

```
+   addition
-   subtraction
*   multiplication
/   division (float)
//  floor division (drops decimals)
%   remainder
**  exponent
```

## Examples

```
10 + 3   # 13
10 - 3   # 7
10 * 3   # 30
10 / 3   # 3.333...
10 // 3  # 3
10 % 3   # 1
2 ** 3   # 8
```

# Comparison Operators

```
x == y   # equal
x != y   # not equal
x > y
x < y
x >= y
x <= y
```

Example:

```
score = 85
score >= 90   # False
score < 100   # True
```

---

# Logical Operators

```
and   # both conditions True
or    # at least one True
not   # reverses boolean
```

Example:

```
age = 20
has_id = True

age >= 18 and has_id   # True
age < 18 or has_id     # True
not has_id             # False
```

---

# Assignment Operators

Shorthand updates:

```
x = 5
x += 2   # 7
x -= 1   # 6
x *= 3   # 18
x /= 2   # 9.0
```

Used often in loops and accumulations.

---

# Type Casting (Conversion)

Convert between types:

```
int("10")      # 10
float("3.14")  # 3.14
str(123)       # "123"
bool("hi")     # True
bool("")       # False
```

Used for:

- numeric input
- formatting
- arithmetic

---

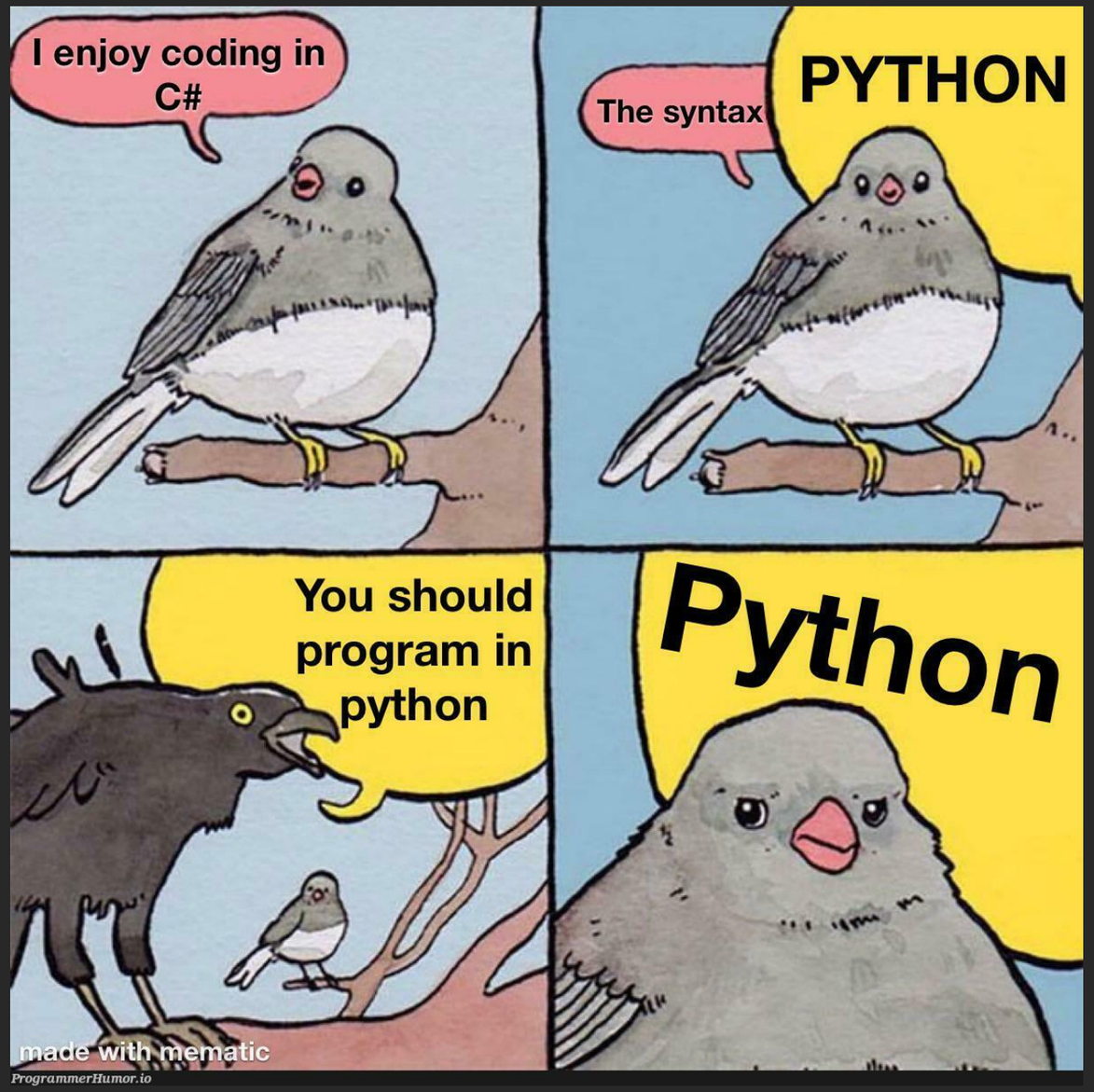
# Example

```python
name = input("Enter your name: ")
age = int(input("Enter your age: "))

next_age = age + 1
is_adult = age >= 18
message = f"{name}, next year you will be {next_age}."

print(message)
print("Adult status:", is_adult)
```

# Mini Task

Create a script that:

1. Asks for two numbers
2. Converts them to integers
3. Calculates:
   - sum
   - difference
   - product
4. Prints results



# Thanks!

By ElnurBDa

# Python Basics Practice

Python Course · Module 2 Practice

ElnurBDa

# Warm-Up — Quick Questions

Without running code, guess the results:

```
1 + 2 * 3
(1 + 2) * 3
10 / 3
10 // 3
10 % 3
```

Then open a Python REPL or script file and **verify** your answers.

Why? Getting a “feel” for operator behavior makes later code easier to read.

---

# Practice 1 — Simple Calculator

Task:

Create `calculator.py` inside `projects/module2_basics/` that:

1. Asks the user for two numbers.
2. Converts them to `float`.
3. Computes and prints:
   - sum
   - difference
   - product
   - quotient (second not zero)

Example interaction:

```
Enter first number: 10
Enter second number: 3
Sum: 13.0
Difference: 7.0
Product: 30.0
Quotient: 3.3333333333
```

Try adding **rounding** with `round(value, 2)`.

---

# Practice 2 — Truth Tables with and / or / not

In `logic_table.py`, without using user input:

1. Create variables `a = True`, `b = False`.
2. Print a simple truth table:

```
a      b      a and b   a or b   not a
True   False  False     True     False
...
```

Use **f-strings** for clean formatting.

---

# Practice 3 — Simple Tip Calculator

tip.py:

```
1. Ask for bill amount (float).
2. Ask for desired tip percentage (e.g., 10, 15, 20).
3. Compute tip and total.
4. Print results with 2 decimal places.
```

---

# Mini Project — Profile Summary

Create profile.py that:

1. Asks for:
   - name (str)
   - age (int)
   - city (str)
   - favorite number (int)
2. Uses arithmetic to compute:
   - age next year
   - favorite number squared
3. Prints a nicely formatted multi-line summary using an f-string.

Example output:

```
Hello, Ada!
You live in London and next year you will be 37.
Your favorite number 42 squared is 1764.
```

---

# Python Basics Practice — Solutions

Python Course · Module 2 Practice Solutions

ElnurBDa

# Warm-Up — Quick Questions (Answers)

```
1 + 2 * 3        # 7 (multiplication first: 2*3=6, then 1+6=7)
(1 + 2) * 3      # 9 (parentheses first: 1+2=3, then 3*3=9)
10 / 3           # 3.3333333333333335 (float division)
10 // 3          # 3 (floor division, integer result)
10 % 3           # 1 (remainder: 10 divided by 3 is 3 with remainder 1)
```

**Key takeaway**: Operator precedence matters! Multiplication/division happen before addition/subtraction.

---

# Practice 1 — Simple Calculator (Solution)

**Basic Solution:**

```python
# Get two numbers from user
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

# Calculate operations
sum_result = num1 + num2
difference = num1 - num2
product = num1 * num2

# Print results
print(f"Sum: {sum_result}")
print(f"Difference: {difference}")
print(f"Product: {product}")

# Handle division (check for zero)
if num2 != 0:
    quotient = num1 / num2
    print(f"Quotient: {round(quotient, 2)}")
else:
    print("Quotient: Cannot divide by zero!")
```

**With Rounding:**

```python
# Get two numbers from user
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

# Calculate operations
sum_result = round(num1 + num2, 2)
difference = round(num1 - num2, 2)
product = round(num1 * num2, 2)

# Print results
print(f"Sum: {sum_result}")
print(f"Difference: {difference}")
print(f"Product: {product}")

# Handle division (check for zero)
if num2 != 0:
    quotient = round(num1 / num2, 2)
    print(f"Quotient: {quotient}")
else:
    print("Quotient: Cannot divide by zero!")
```

---

# Practice 2 — Truth Tables (Solution)

**Basic Solution:**

```python
# Create boolean variables
a = True
b = False

# Print header
print(f"{'a':<8} {'b':<8} {'a and b':<12} {'a or b':<12} {'not a':<12}")
print("-" * 60)

# Print first row (a=True, b=False)
print(f"{str(a):<8} {str(b):<8} {str(a and b):<12} {str(a or b):<12} {str
(not a):<12}")

# Print second row (a=True, b=True)
a = True
b = True
print(f"{str(a):<8} {str(b):<8} {str(a and b):<12} {str(a or b):<12} {str
(not a):<12}")

# Print third row (a=False, b=True)
a = False
b = True
print(f"{str(a):<8} {str(b):<8} {str(a and b):<12} {str(a or b):<12} {str
(not a):<12}")

# Print fourth row (a=False, b=False)
a = False
b = False
print(f"{str(a):<8} {str(b):<8} {str(a and b):<12} {str(a or b):<12} {str
(not a):<12}")
```

**More Elegant Solution** (using a loop):

```python
# Print header
print(f"{'a':<8} {'b':<8} {'a and b':<12} {'a or b':<12} {'not a':<12}")
print("-" * 60)

# Generate all combinations
combinations = [
    (True, False),
    (True, True),
    (False, True),
    (False, False)
]

# Print each row
for a, b in combinations:
    print(f"{str(a):<8} {str(b):<8} {str(a and b):<12} {str(a or b):<12} {
str(not a):<12}")
```

# Practice 3 – Simple Tip Calculator (Solution)

**Basic Solution**:

```python
# Get bill amount
bill = float(input("Enter bill amount: "))

# Get tip percentage
tip_percent = float(input("Enter tip percentage (e.g., 10, 15, 20): "))

# Calculate tip amount
tip_amount = bill * (tip_percent / 100)

# Calculate total
total = bill + tip_amount

# Print results with 2 decimal places
print(f"Bill: ${bill:.2f}")
print(f"Tip ({tip_percent}%): ${tip_amount:.2f}")
print(f"Total: ${total:.2f}")
```

**Alternative Approach** (more detailed):

```python
# Get bill amount
bill = float(input("Enter bill amount: $"))

# Get tip percentage
tip_percent = float(input("Enter tip percentage: "))

# Calculate tip and total
tip_amount = round(bill * (tip_percent / 100), 2)
total = round(bill + tip_amount, 2)

# Print formatted results
print(f"\nBill: ${bill:.2f}")
print(f"Tip ({tip_percent}%): ${tip_amount:.2f}")
print(f"Total: ${total:.2f}")
```

---

# Mini Project — Profile Summary (Solution)

**Basic Solution**:

```python
# Get user information
name = input("Enter your name: ")
age = int(input("Enter your age: "))
city = input("Enter your city: ")
favorite_number = int(input("Enter your favorite number: "))

# Calculate age next year
age_next_year = age + 1

# Calculate favorite number squared
number_squared = favorite_number ** 2

# Print formatted summary
print(f"\nHello, {name}!")
print(f"You live in {city} and next year you will be {age_next_year}.")
print(f"Your favorite number {favorite_number} squared is {number_squared}.
")
```

**Alternative Approach** (more detailed formatting):

```python
# Get user information
name = input("Enter your name: ")
age = int(input("Enter your age: "))
city = input("Enter your city: ")
favorite_number = int(input("Enter your favorite number: "))

# Calculations
age_next_year = age + 1
number_squared = favorite_number ** 2

# Print nicely formatted summary
print("\n" + "=" * 40)
print(f"Hello, {name}!")
print(f"You live in {city} and next year you will be {age_next_year}.")
print(f"Your favorite number {favorite_number} squared is {number_squared}.
")
print("=" * 40)
```

---

# Data Structures

Python Course · Module 3

ElnurBDa

# Data Structures — Overview

Python provides multiple built-in structures for organizing data:

- **Lists**
- **Strings**
- **Tuples**
- **Sets**
- **Dictionaries**
- Choosing the right structure

Each structure has unique properties and use-cases.

---

# Lists

Lists store **ordered, changeable** sequences.

```python
numbers = [10, 20, 30]

# Operations:
numbers.append(40)
numbers[1] = 99
numbers.remove(10)
print(numbers)

# List indexing:
print(numbers[0])
print(numbers[-1])
print(numbers[1:3])
```

[finished]

```
[99, 30, 40]
99
40
[30, 40]
```

# List Comprehensions

List comprehensions are a **compact way** to build new lists from existing data.

Basic pattern:

```
numbers = [1, 2, 3, 4, 5]

squares = [n * n for n in numbers]
evens = [n for n in numbers if n % 2 == 0]

print("squares:", squares)
print("evens:", evens)
```

——————————— [finished] ———————————

```
squares: [1, 4, 9, 16, 25]
evens: [2, 4]
```

They are great for:

- simple transformations
- filtering values
- avoiding manual `for` + `.append()` boilerplate

Rule of thumb: if the logic fits in **one short line**, a comprehension is OK; otherwise, use a normal loop for readability.

---

# List Characteristics

Advantages:

- Mutable
- Dynamic size
- Supports duplicates

Use-cases:

- Collections
- Queues
- Storing ordered items

---

# Strings

Strings represent **text**.

Properties:

- Ordered
- Immutable
- Indexable
- Iterable

```python
s = "Hello"

a = s[0]       # first character
b = s[-1]      # last character
c = s.upper()  # all upper chars
d = len(s)     # Length of the string

print(a, b, c, d)
```

[finished]

```
H o HELLO 5
```

# Tuples

Tuples store **ordered, immutable** sequences.

```
point = (10, 20)
```

Benefits:

- Fast
- Secure from modification
- Memory-efficient

Tuple unpacking:

```
x, y = point
```

---

# Tuple vs List

- Need modification → **List**
- Need fixed, reliable structure → **Tuple**
- Performance required → **Tuple**

---

# Sets

Sets store **unordered, unique** elements.

```
colors = {"red", "blue", "green"}
```

Operations:

```
colors.add("yellow")
colors.remove("red")
```

Useful for:

- membership checks
- removing duplicates
- mathematical operations

---

# Set Operations

```python
a = {1, 2, 3}
b = {3, 4, 5}

x1 = a | b    # union
x2 = a & b    # intersection
x3 = a - b    # difference
x4 = a ^ b    # symmetric difference

print(x1, x2, x3, x4)
```

[finished]

```
{1, 2, 3, 4, 5} {3} {1, 2} {1, 2, 4, 5}
```

# Dictionaries

Dictionaries store **key : value** pairs.

```python
user = {
    "name": "Alice",
    "age": 25,
    "city": "Baku"
}

# Access:
n = user["name"]
print(n)
print()

# Modify:
user["age"] = 26
user["country"] = "Azerbaijan"
print(user)
```

[finished]

```
Alice

{'name': 'Alice', 'age': 26, 'city': 'Baku', 'country':
'Azerbaijan'}
```

# Dictionary Operations

```python
user = {
    "name": "Alice",
    "age": 25,
    "city": "Baku"
}

a = user.keys()
b = user.values()
c = user.items()
d = "city" in user  # check key existence
e = "Baku" in user

print(a)
print(b)
print(c)
print(d)
print(e)
```

[finished]

```
dict_keys(['name', 'age', 'city'])
dict_values(['Alice', 25, 'Baku'])
dict_items([('name', 'Alice'), ('age', 25), ('city',
'Baku')])
True
False
```

Dictionaries are ideal for structured data.

# Example

```python
student = {
    "name": "Elnur",
    "scores": [90, 85, 97],
    "passed": True
}

average = sum(student["scores"]) / len(student["scores"])

print(student["name"], "average:", average)
```

[finished]

```
Elnur average: 90.66666666666667
```

# Choosing the Right Data Structure

| Type | Ordered | Mutable | Unique | Best For |
|---|---|---|---|---|
| **String** | Yes | No | N/A | Text |
| **List** | Yes | Yes | No | General collections |
| **Tuple** | Yes | No | No | Fixed data |
| **Set** | No | Yes | Yes | Unique items |
| **Dict** | Keys no | Yes | Keys unique | Structured data |

# Mini Task

Create a script that:

1. Stores student info in a dictionary
2. Contains: name, age, list of grades
3. Calculates average grade
4. Prints a formatted summary

---

# Thanks!

By ElnurBDa

# Control Flow

Python Course · Module 4

ElnurBDa

# Control Flow — Overview

Control flow decides **how a program behaves**:

- Conditional statements
- Logical operators
- Loops (`for`, `while`)
- Loop control (`break`, `continue`, `pass`)

This module teaches how Python makes decisions and repeats actions.

---

# Conditional Statements

Basic structure:

```
if condition:
    block
```

A condition must evaluate to **True** or **False**.

Example:

```
age = 20
if age >= 18:
    print("Adult")
```

[finished]

```
Adult
```

# if / else

```
if condition:
    action_if_true
else:
    action_if_false
```

Example:

```
temperature = 10

if temperature > 15:
    print("Warm")
else:
    print("Cold")
```

[finished]

```
Cold
```

# if / elif / else

Used when multiple conditions must be checked in order.

```python
grade = "?"
score = 75

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "D"

print(grade)
```

[finished]

```
C
```

# Logical Operators in Conditions

```
and   # both True
or    # one True
not   # invert
```

Examples:

```
age = 20
has_id = True

age >= 18 and has_id
age < 18 or has_id
not has_id
```

---

# Nested Conditions

```python
age = 25
citizen = True

if age >= 18:
    if citizen:
        print("Eligible")
    else:
        print("Not eligible")
```

[finished]

```
Eligible
```

Avoid deep nesting when possible.

# for Loop — Basics

Iterates over a sequence:

```python
for x in [1, 2, 3]:
    print(x)
```

[finished]

```
1
2
3
```

Strings:

```python
for char in "Python":
    print(char)
```

[finished]

```
P
y
t
h
o
n
```

# for Loop with range()

```python
for i in range(5):
    print(i)
```

[finished]

```
0
1
2
3
4
```

Custom range:

```python
for i in range(2, 10, 2):
    print(i)
```

[finished]

```
2
4
6
8
```

# while Loop

Repeats while a condition is **True**.

```
count = 0
while count < 5:
    print(count)
    count += 1
```

[finished]

```
0
1
2
3
4
```

# Loop Control — break

Stops the loop immediately.

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

[finished]

```
0
1
2
3
4
```

# Loop Control — continue

Skips current iteration.

```
for i in range(5):
    if i % 2 == 0:
        continue
    print(i)
```

[finished]

```
1
3
```

# Loop Control — pass

pass does nothing — placeholder.

```
for i in range(3):
    pass
```

[finished]

# Loop Patterns & Comprehensions

Often we use loops to **build new lists** from old ones.

Classic pattern:

```
nums = [1, 2, 3, 4, 5]
evens = []

for n in nums:
    if n % 2 == 0:
        evens.append(n)

print("evens:", evens)
```

——————————— [finished] ———————————

```
evens: [2, 4]
```

Same logic using a **list comprehension** (from Module 3):

```
nums = [1, 2, 3, 4, 5]
evens = [n for n in nums if n % 2 == 0]
print("evens:", evens)
```

——————————— [finished] ———————————

```
evens: [2, 4]
```

Use whichever is **clearer** to you and your team; for multiple complex steps, prefer the classic loop.

---

# Example

```python
total = 0

for i in range(1, 6):
    if i % 2 == 0:
        total += i
    else:
        continue

print("Sum of even numbers:", total)
```

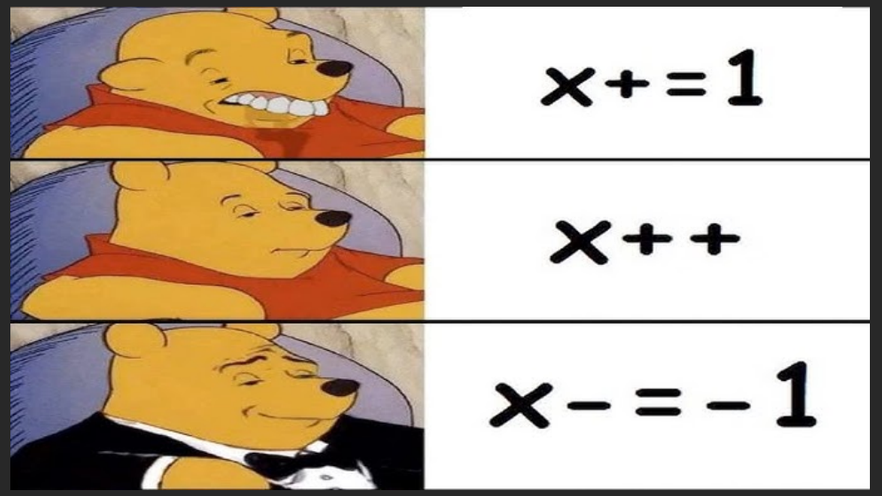
[finished]

```
Sum of even numbers: 6
```

# Mini Task

Write a script that:

1. Asks the user for a number
2. Prints all numbers from 1 to that number
3. Prints only **even numbers**
4. Skips odd numbers using continue
5. Stops entirely if number exceeds 100 using break



# Thanks!

By ElnurBDa

# Control Flow Practice

Python Course · Module 4 Practice

ElnurBDa

# Practice 1 — Password Validator

Create `password_validator.py` that:

1. Asks the user to enter a password.
2. Validates the password using these rules:
   - At least 8 characters long
   - Contains at least one uppercase letter
   - Contains at least one lowercase letter
   - Contains at least one digit
   - Contains at least one special character (`!@#$%^&*`)
3. Uses loops and conditionals to check each requirement.
4. Prints clear feedback for each failed rule.
5. Only accepts the password if **all** rules pass.

Example interaction:

```
Enter password: abc123
Password too short (minimum 8 characters)
Missing uppercase letter
Missing special character
Password rejected!

Enter password: SecurePass123!
Password accepted!
```

---

# Practice 2 — Shopping Cart Manager

Create `shopping_cart.py` that:

1. Uses a **dictionary** to store items and their prices:

```
cart = {"apple": 1.50, "bread": 2.00, "milk": 3.50}
```

2. Implements a menu system using a `while` loop:
   - "1. Add item"
   - "2. Remove item"
   - "3. View cart"
   - "4. Calculate total"
   - "5. Exit"
3. For "Add item": asks for item name and price, adds to dictionary.
4. For "Remove item": asks for item name, removes if exists.
5. For "View cart": displays all items with prices formatted to 2 decimals.
6. For "Calculate total": sums all prices and prints the total.
7. Uses `break` to exit the loop when user chooses "5".

Example interaction:

```
Menu:
1. Add item
2. Remove item
3. View cart
4. Calculate total
5. Exit
Choice: 1
Item name: orange
Price: 2.25
Item added!

Choice: 3
Cart:
apple: $1.50
bread: $2.00
milk: $3.50
orange: $2.25

Choice: 4
Total: $9.25
```

---

# Practice 3 — Number Pattern Generator

Create `pattern_generator.py` that:

1. Asks the user for a number `n` (between 1 and 20).
2. Validates the input (must be between 1 and 20).
3. Uses nested `for` loops to generate patterns:
3. **Pattern A** (right-aligned triangle):

```
*
**
***
****
*****
```

**Pattern B** (number pyramid):

```
1
22
333
4444
55555
```

4. Prints both patterns one after another using nested loops.
5. Uses `range()` for loop control.

---

# Practice 4 — Student Grade Analyzer

Create grade_analyzer.py that:

1. Uses a **dictionary** to store student names and their list of grades:

```
students = {
      "Alice": [85, 90, 78],
      "Bob": [92, 88, 95],
      "Charlie": [70, 75, 80]
}
```

2. Loops through each student in the dictionary.
3. For each student:
   - Calculates their average grade (sum of grades divided by count)
   - Determines their letter grade using if/elif/else:
     - "A" for 90+, "B" for 80-89, "C" for 70-79, "D" for 60-69, "F" for <60
   - Prints a formatted report line
4. Uses a loop to find and print which students have an average above 85.
5. Use list comprehensions where appropriate (e.g., filtering high scores).

Example output:

```
Student Report:
==============
Alice: Average = 84.33, Grade = B
Bob: Average = 91.67, Grade = A
Charlie: Average = 75.00, Grade = C

High Performers (Average > 85):
- Bob
```

---

# Practice 5 — Text Statistics Tool

Create `text_stats.py` that:

1. Asks the user to enter a paragraph of text (multiple sentences).
2. Uses loops and conditionals to analyze the text:
   - Count words (split by spaces)
   - Count sentences (ends with `.`, `!`, `?`)
   - Count vowels (a, e, i, o, u, case-insensitive)
   - Find the longest word
   - Count characters (with and without spaces)
3. Uses loops to iterate through characters and words.
4. Uses conditionals to check for sentence endings and vowels.
5. Stores results in variables and prints a formatted statistics report.

Example:

```
Enter text: Hello world! How are you? I am fine.
========================================
Text Statistics:
========================================
Total words: 7
Total sentences: 3
Total vowels: 12
Longest word: world
Characters (with spaces): 33
Characters (without spaces): 30
```

---

# Practice 6 — Set Operations & Data Filtering

Create `data_filter.py` that:

1. Creates two sets of user IDs:

```
active_users = {101, 102, 103, 104, 105}
premium_users = {103, 104, 106, 107}
```

2. Uses set operations to find:
   - Users who are both active AND premium (intersection)
   - All unique users (union)
   - Active users who are NOT premium (difference)
   - Users who are in one set but not both (symmetric difference)
3. Creates a dictionary `users` with structure: `{user_id: {"name": str, "age": int}}`
4. Uses a loop with conditionals to filter users by age (between 20 and 28 inclusive).
5. Stores filtered user IDs in a list and prints formatted results for all operations.

Example:

```
users = {
    101: {"name": "Alice", "age": 25},
    102: {"name": "Bob", "age": 30},
    103: {"name": "Charlie", "age": 22}
}
```

Example output:

```
Set Operations:
===============
Active AND Premium: {103, 104}
All Users: {101, 102, 103, 104, 105, 106, 107}
Active but NOT Premium: {101, 102, 105}
Symmetric Difference: {101, 102, 105, 106, 107}

Age Filter (20-28):
===================
Users aged 20-28: [101, 103]
```

---

# Mini Project — Personal Task Manager

Create task_manager.py that combines multiple concepts:

1. Use a **list of dictionaries** to store tasks:

```
tasks = [
      {"id": 1, "title": "Learn Python", "priority": "high", "completed": False},
      {"id": 2, "title": "Exercise", "priority": "medium", "completed": True}
]
```

2. Implement a menu-driven system with a while loop:
   - "1. Add task"
   - "2. Mark task as complete"
   - "3. View all tasks"
   - "4. View incomplete tasks only"
   - "5. View tasks by priority"
   - "6. Delete task"
   - "7. Statistics"
   - "8. Exit"
3. For each menu option, use conditionals and loops to:
   - Add new tasks to the list
   - Mark tasks as complete by ID
   - Display tasks (all, incomplete only, or by priority)
   - Delete tasks by ID
   - Calculate and display statistics
4. Statistics should show:
   - Total tasks
   - Completed vs incomplete count (use loops to count)
   - Tasks by priority (use a dictionary and loops to count)
5. Use continue to skip invalid menu choices.
6. Use break to exit when user chooses "8".
7. Use conditionals to check if task ID exists before operations.

---

# Control Flow Practice — Solutions

Python Course · Module 4 Practice Solutions

ElnurBDa

# Practice 1 – Password Validator (Solution)

```python
# Get password from user
password = input("Enter password: ")

# Initialize validation flags
has_length = False
has_uppercase = False
has_lowercase = False
has_digit = False
has_special = False

# Check length
if len(password) >= 8:
    has_length = True
else:
    print("Password too short (minimum 8 characters)")

# Check for uppercase letter
for char in password:
    if char >= 'A' and char <= 'Z':
        has_uppercase = True
        break

if not has_uppercase:
    print("Missing uppercase letter")

# Check for lowercase letter
for char in password:
    if char >= 'a' and char <= 'z':
        has_lowercase = True
        break

if not has_lowercase:
    print("Missing lowercase letter")
# ...
```

```python
# ...
# Check for digit
for char in password:
    if char >= '0' and char <= '9':
        has_digit = True
        break

if not has_digit:
    print("Missing digit")

# Check for special character
special_chars = "!@#$%^&*"
has_special = False
for char in password:
    if char in special_chars:
        has_special = True
        break

if not has_special:
    print("Missing special character")

# Final validation
if has_length and has_uppercase and has_lowercase and has_digit and
has_special:
    print("Password accepted!")
else:
    print("Password rejected!")
```

# Practice 2 — Shopping Cart Manager (Solution)

```python
# Initialize cart dictionary
cart = {"apple": 1.50, "bread": 2.00, "milk": 3.50}

# Main menu loop
while True:
    print("\nMenu:")
    print("1. Add item")
    print("2. Remove item")
    print("3. View cart")
    print("4. Calculate total")
    print("5. Exit")

    choice = input("Choice: ")

    if choice == "1":
        # Add item
        item_name = input("Item name: ")
        price = float(input("Price: "))
        cart[item_name] = price
        print("Item added!")

    elif choice == "2":
        # Remove item
        item_name = input("Item name: ")
        if item_name in cart:
            del cart[item_name]
            print("Item removed!")
        else:
            print("Item not found in cart")

# ...
```

```python
# ...
    elif choice == "3":
        # View cart
        print("\nCart:")
        for item, price in cart.items():
            print(f"{item}: ${price:.2f}")

    elif choice == "4":
        # Calculate total
        total = 0
        for price in cart.values():
            total += price
        print(f"Total: ${total:.2f}")

    elif choice == "5":
        # Exit
        print("Goodbye!")
        break

    else:
        print("Invalid choice. Please try again.")
```

# Practice 3 – Number Pattern Generator (Solution)

**Solution**:

```python
# Get number from user
n = int(input("Enter a number (1-20): "))

# Validate input
if n < 1 or n > 20:
    print("Number must be between 1 and 20")
else:
    # Pattern A: Right-aligned triangle
    print("\nPattern A:")
    for i in range(1, n + 1):
        for j in range(i):
            print("*", end="")
        print()

    # Pattern B: Number pyramid
    print("\nPattern B:")
    for i in range(1, n + 1):
        for j in range(i):
            print(i, end="")
        print()
```

**Alternative Solution** (using string multiplication):

```python
# Get number from user
n = int(input("Enter a number (1-20): "))

# Validate input
if n < 1 or n > 20:
    print("Number must be between 1 and 20")
else:
    # Pattern A: Right-aligned triangle
    print("\nPattern A:")
    for i in range(1, n + 1):
        print("*" * i)

    # Pattern B: Number pyramid
    print("\nPattern B:")
    for i in range(1, n + 1):
        print(str(i) * i)
```

---

# Practice 4 – Student Grade Analyzer (Solution)

**Solution**:

```python
# Initialize students dictionary
students = {
    "Alice": [85, 90, 78],
    "Bob": [92, 88, 95],
    "Charlie": [70, 75, 80]
}

# Print header
print("Student Report:")
print("=" * 15)

# Loop through each student
for name, grades in students.items():
    # Calculate average
    total = 0
    for grade in grades:
        total += grade
    average = total / len(grades)

    # Determine letter grade
    if average >= 90:
        letter_grade = "A"
    elif average >= 80:
        letter_grade = "B"
    elif average >= 70:
        letter_grade = "C"
    elif average >= 60:
        letter_grade = "D"
    else:
        letter_grade = "F"

    # Print formatted report
    print(f"{name}: Average = {average:.2f}, Grade = {letter_grade}")

# Find high performers (average > 85)
print("\nHigh Performers (Average > 85):")
high_performers = []
for name, grades in students.items():
    total = 0
    for grade in grades:
        total += grade
    average = total / len(grades)
    if average > 85:
        high_performers.append(name)

# Print high performers
for name in high_performers:
    print(f"- {name}")
```

**Alternative Solution** (more compact structure):

```python
# Initialize students dictionary
students = {
    "Alice": [85, 90, 78],
    "Bob": [92, 88, 95],
    "Charlie": [70, 75, 80]
}

# Print header
print("Student Report:")
print("=" * 15)

# Loop through each student
for name, grades in students.items():
    # Calculate average manually
    total = 0
    for grade in grades:
        total += grade
    average = total / len(grades)

    # Determine letter grade
    if average >= 90:
        letter_grade = "A"
    elif average >= 80:
        letter_grade = "B"
    elif average >= 70:
        letter_grade = "C"
    elif average >= 60:
        letter_grade = "D"
    else:
        letter_grade = "F"

    # Print formatted report
    print(f"{name}: Average = {average:.2f}, Grade = {letter_grade}")

# Find high performers
print("\nHigh Performers (Average > 85):")
high_performers = []
for name, grades in students.items():
    total = 0
    for grade in grades:
        total += grade
    average = total / len(grades)
    if average > 85:
        high_performers.append(name)

# Print high performers
for name in high_performers:
    print(f"- {name}")
```

# Practice 5 — Text Statistics Tool (Solution)

**Solution** (manual word splitting):

```python
# Get text from user
text = input("Enter text: ")

# Count words (manual splitting by spaces)
words = []
current_word = ""
for char in text:
    if char == ' ':
        if current_word != "":
            words.append(current_word)
            current_word = ""
    else:
        current_word += char
# Add last word if text doesn't end with space
if current_word != "":
    words.append(current_word)

word_count = len(words)

# Count sentences (ends with ., !, ?)
sentence_count = 0
for char in text:
    if char == '.' or char == '!' or char == '?':
        sentence_count += 1

# Count vowels (case-insensitive)
vowel_count = 0
vowels = "aeiouAEIOU"
for char in text:
    if char in vowels:
        vowel_count += 1

# Find longest word
longest_word = ""
for word in words:
    if len(word) > len(longest_word):
        longest_word = word

# Count characters
char_count_with_spaces = len(text)
char_count_without_spaces = 0
for char in text:
    if char != ' ':
        char_count_without_spaces += 1

# Print statistics
print("=" * 40)
print("Text Statistics:")
print("=" * 40)
print(f"Total words: {word_count}")
print(f"Total sentences: {sentence_count}")
print(f"Total vowels: {vowel_count}")
print(f"Longest word: {longest_word}")
print(f"Characters (with spaces): {char_count_with_spaces}")
```

**Alternative Solution** (using basic string operations):

```python
# Get text from user
text = input("Enter text: ")

# Count words by counting spaces + 1 (with edge case handling)
word_count = 1  # Start with 1 word
for i in range(len(text)):
    if text[i] == ' ':
        # Check if next char is not a space (to avoid counting multiple
spaces)
        if i + 1 < len(text) and text[i + 1] != ' ':
            word_count += 1
# Handle empty text
if len(text) == 0:
    word_count = 0

# Count sentences
sentence_endings = ".!?"
sentence_count = 0
for char in text:
    if char in sentence_endings:
        sentence_count += 1

# Count vowels
vowels = "aeiouAEIOU"
vowel_count = 0
for char in text:
    if char in vowels:
        vowel_count += 1

# Find longest word (build words manually)
longest_word = ""
current_word = ""
for char in text:
    if char == ' ':
        if len(current_word) > len(longest_word):
            longest_word = current_word
        current_word = ""
    else:
        current_word += char
# Check last word
if len(current_word) > len(longest_word):
    longest_word = current_word

# Count characters
char_count_with_spaces = len(text)
char_count_without_spaces = 0
for char in text:
    if char != ' ':
        char_count_without_spaces += 1

# Print statistics
print("=" * 40)
print("Text Statistics:")
print("=" * 40)
```

# Practice 6 – Set Operations & Data Filtering (Solution)

**Solution**:

```python
# Create sets
active_users = {101, 102, 103, 104, 105}
premium_users = {103, 104, 106, 107}

# Set operations
active_and_premium = active_users & premium_users  # intersection
all_users = active_users | premium_users  # union
active_not_premium = active_users - premium_users  # difference
symmetric_diff = active_users ^ premium_users  # symmetric difference

# Print set operations
print("Set Operations:")
print("=" * 15)
print(f"Active AND Premium: {active_and_premium}")
print(f"All Users: {all_users}")
print(f"Active but NOT Premium: {active_not_premium}")
print(f"Symmetric Difference: {symmetric_diff}")

# Create users dictionary
users = {
    101: {"name": "Alice", "age": 25},
    102: {"name": "Bob", "age": 30},
    103: {"name": "Charlie", "age": 22},
    104: {"name": "Diana", "age": 27},
    105: {"name": "Eve", "age": 19}
}

# Filter users by age (20-28 inclusive)
filtered_users = []
for user_id, user_info in users.items():
    age = user_info["age"]
    if age >= 20 and age <= 28:
        filtered_users.append(user_id)

# Print filtered results
print("\nAge Filter (20-28):")
print("=" * 15)
print(f"Users aged 20-28: {filtered_users}")
```

**Alternative Solution** (using list comprehension):

```python
# Create sets
active_users = {101, 102, 103, 104, 105}
premium_users = {103, 104, 106, 107}

# Set operations
active_and_premium = active_users & premium_users
all_users = active_users | premium_users
active_not_premium = active_users - premium_users
symmetric_diff = active_users ^ premium_users

# Print set operations
print("Set Operations:")
print("=" * 15)
print(f"Active AND Premium: {active_and_premium}")
print(f"All Users: {all_users}")
print(f"Active but NOT Premium: {active_not_premium}")
print(f"Symmetric Difference: {symmetric_diff}")

# Create users dictionary
users = {
    101: {"name": "Alice", "age": 25},
    102: {"name": "Bob", "age": 30},
    103: {"name": "Charlie", "age": 22},
    104: {"name": "Diana", "age": 27},
    105: {"name": "Eve", "age": 19}
}

# Filter users by age using list comprehension
filtered_users = [
    user_id for user_id, user_info in users.items()
    if user_info["age"] >= 20 and user_info["age"] <= 28
]

# Print filtered results
print("\nAge Filter (20-28):")
print("=" * 15)
print(f"Users aged 20-28: {filtered_users}")
```

---

# Mini Project – Personal Task Manager (Solution)

```python
# Initialize tasks list
tasks = [
    {"id": 1, "title": "Learn Python", "priority
": "high", "completed": False},
    {"id": 2, "title": "Exercise", "priority": "
medium", "completed": True}
]

# Main menu loop
while True:
    print("\nMenu:")
    print("1. Add task")
    print("2. Mark task as complete")
    print("3. View all tasks")
    print("4. View incomplete tasks only")
    print("5. View tasks by priority")
    print("6. Delete task")
    print("7. Statistics")
    print("8. Exit")

    choice = input("\nChoice: ")

    if choice == "1":
        # Add task
        task_id = int(input("Enter task ID: "))
        title = input("Enter task title: ")
        priority = input("Enter priority
(high/medium/low): ")
        tasks.append({
            "id": task_id,
            "title": title,
            "priority": priority,
            "completed": False
        })
        print("Task added!")

    elif choice == "2":
        # Mark task as complete
        task_id = int(input("Enter task ID to
mark complete: "))
        found = False
        for task in tasks:
            if task["id"] == task_id:
                task["completed"] = True
                print("Task marked as complete!
")
                found = True
                break
        if not found:
            print("Task not found!")

    elif choice == "3":
        # View all tasks
        print("\nAll Tasks:")
        print("-" * 40)
        for task in tasks:
            status = "✓" if task["completed"]
else "✗"
            print(f"ID: {task['id']}, Title:
{task['title']}, "
                  f"Priority: {task['priority
']}, Completed: {status}")

    elif choice == "4":
        # View incomplete tasks only
        print("\nIncomplete Tasks:")
        print("-" * 40)
        for task in tasks:
            if not task["completed"]:
                print(f"ID: {task['id']},
Title: {task['title']}, "
                      f"Priority: {task['
priority']}")

    elif choice == "5":
        # View tasks by priority
        priority_filter = input("Enter
priority (high/medium/low): ")
        print(f"\nTasks with priority '
{priority_filter}':")
        print("-" * 40)
        for task in tasks:
            if task["priority"] ==
priority_filter:
                status = "✓" if task["
completed"] else "✗"
                print(f"ID: {task['id']},
Title: {task['title']}, "
                      f"Completed: {status}")

    elif choice == "6":
        # Delete task
        task_id = int(input("Enter task ID to
delete: "))
        found = False
        for i in range(len(tasks)):
            if tasks[i]["id"] == task_id:
                tasks.pop(i)
                print("Task deleted!")
                found = True
                break
        if not found:
            print("Task not found!")

    elif choice == "7":
        # Statistics
        total_tasks = len(tasks)
        completed_count = 0
        incomplete_count = 0

        for task in tasks:
            if task["completed"]:
                completed_count += 1
            else:
                incomplete_count += 1

        # Count by priority
        priority_count = {"high": 0, "medium": 0,
"low": 0}
        for task in tasks:
            priority = task["priority"]
            if priority in priority_count:
                priority_count[priority] += 1

        print("\nStatistics:")
        print("=" * 40)
        print(f"Total tasks: {total_tasks}")
        print(f"Completed: {completed_count}")
        print(f"Incomplete: {incomplete_count}")
        print("\nTasks by Priority:")
        for priority, count in priority_count.
items():
            print(f"  {priority}: {count}")

    elif choice == "8":
        # Exit
        print("Goodbye!")
        break

    else:
        # Invalid choice
        print("Invalid choice. Please try again.
")
        continue
```

# Functions & Modular Code

Python Course · Module 5

ElnurBDa

# Functions — Introduction

Functions group reusable code into named blocks.

Benefits:

- Avoid repetition
- Improve readability
- Easier debugging
- Modular structure

Basic structure:

```
def name():
    pass
```

---

# Creating Functions

```
def greet():
    print("Hello!")
```

Call:

```
greet()
```

Functions must be defined **before** calling.

---

# Parameters

```python
def greet(name):
    print("Hello,", name)
```

Multiple parameters:

```python
def add(a, b):
    return a + b
```

---

# Return Values

```python
def square(x):
    return x * x
```

Functions without `return` return `None`.

---

# Multiple Returns

```
def values():
    return 10, 20, 30
```

Unpacking:

```
a, b, c = values()
```

---

# Variable Scope

```python
x = 10      # global

def func():
    y = 5   # local
```

Local variables override global ones inside functions.

---

# global Keyword

```python
count = 0

def inc():
    global count
    count += 1
```

Use sparingly; global state makes code harder to manage.

# Lambda Functions

```
square = lambda x: x * x
```

Often used inline:

```
add = lambda a, b: a + b
```

---

# Higher-Order Functions

```
nums = [1, 2, 3, 4]
squares = list(map(lambda x: x*x, nums))
```

Using normal function:

```
def sq(x): return x*x
```

---

# Modules — Importing

```
import math
math.sqrt(16)
```

Selective:

```
from math import sqrt
```

Rename:

```
import math as m
```

---

# Built-in Modules

Examples:

- math
- random
- os
- sys
- datetime

Example:

```
import random
x = random.randint(1, 10)
```

---

# Creating Your Own Module

File:

```
mymath.py
```

Content:

```
def add(a, b):
    return a + b
```

Usage:

```
import mymath
mymath.add(2, 3)
```

---

# Organizing Code

```
project/
├── main.py
├── utils.py
└── math_ops.py
```

```python
from utils import greet
from math_ops import add
```

# Example

```python
# utils.py
def greet(name):
    return f"Hello {name}"

# math_ops.py
def multiply(a, b):
    return a * b

# main.py
from utils import greet
from math_ops import multiply

print(greet("Elnur"))
print(multiply(4, 5))
```

# Mini Task

Create a project with:

1. main.py
2. converter.py with:
   - km_to_miles(km)
   - celsius_to_fahrenheit(c)
3. Import into main.py
4. Ask user input and print converted values

---

# Objects & Classes — Why They Matter

Functions are perfect for **single actions**. Classes help when you need to model **entities** (users, sensors, tickets) that carry both data and behavior.

- **Object** = “thing” with attributes + methods
- **Class** = blueprint describing what every object of that type should have

Analogy:

- Class = recipe for User
- Object = actual user baked from that recipe

Use classes when you keep passing the same fields around or when multiple behaviors belong to the same piece of data.

---

# Defining a Class

```python
class User:
    def __init__(self, username, email):
        self.username = username
        self.email = email
        self.active = True

    def deactivate(self):
        self.active = False

user = User("alice", "alice@example.com")
print(user.username, user.active)
user.deactivate()
print(user.active)
```

[finished]

```
alice True
False
```

Notes:

- `__init__` runs every time you call `User(...)`
- `self` points to the current object so you can store/read attributes
- Methods are normal functions defined inside the class

# Attributes vs Methods

| Concept | Purpose | Example |
|---|---|---|
| Attribute | Data stored on the object | `user.email`, `user.active` |
| Method | Action using that data | `user.deactivate()` |
| Class attr | Shared for all instances | `class User: role = "student"` |

Guidelines:

- Keep attributes small and descriptive
- Use methods when behavior depends on the object's state
- Return values from methods when you need reusable data (`describe()`)

---

# Classes vs Simple Functions

```
Prefer simple functions when:
  - Logic is stateless
  - You just manipulate inputs/outputs quickly
  - The helper stands on its own (e.g., convert units)

Prefer classes when:
  - You repeatedly use the same bundle of data
  - Multiple behaviors belong to that bundle
  - You want to hide implementation details behind a clean API
```

You can mix both patterns: functions orchestrate flows, classes encapsulate objects.

---

# Mini Task — Task Manager OOP

Inside `projects/task_manager/`:

1. Create `task.py` with a `Task` class containing `title`, `completed`, `priority="normal"`, and methods `mark_done()` plus `describe()` returning a formatted summary.
2. Create `main.py` that instantiates several tasks, stores them in a list, and prints each description.
3. Stretch goal: add a `TaskList` class with `add_task()`, `list_pending()` to practice classes working together.

This OOP layer will make it easier to structure file, database, and web projects in upcoming modules.

---

# Thanks!

By ElnurBDa

# Functions & Modular Code Practice

Python Course · Module 5 Practice

ElnurBDa

# Practice 1 — Function Library

Create a project with the following structure:

```
practice1_functions/
├── main.py
└── math_utils.py
```

**Requirements:**

1. In `math_utils.py`, create these functions:
   - `calculate_area(length, width)` - returns the area of a rectangle
   - `calculate_perimeter(length, width)` - returns the perimeter of a rectangle
   - `is_even(number)` - returns `True` if number is even, `False` otherwise
   - `factorial(n)` - calculates and returns the factorial of n ($n! = n \times (n-1) \times \ldots \times 1$)
   - `power(base, exponent)` - returns base raised to the power of exponent
2. In `main.py`:
   - Import all functions from `math_utils`
   - Ask the user for two numbers (length and width)
   - Call `calculate_area()` and `calculate_perimeter()` and display results
   - Ask the user for a number and check if it's even using `is_even()`
   - Ask the user for a number and calculate its factorial
   - Ask the user for base and exponent, then calculate power

**Example interaction:**

```
Enter length: 5
Enter width: 3
Area: 15
Perimeter: 16

Enter a number: 8
Is even: True

Enter a number for factorial: 5
Factorial of 5: 120

Enter base: 2
Enter exponent: 8
2^8 = 256
```

---

# Practice 2 — Text Processing Functions

Create `text_processor.py` that contains:

1. **Functions to create:**
   - `count_words(text)` - returns the number of words in text
   - `count_vowels(text)` - returns the number of vowels (a, e, i, o, u, case-insensitive)
   - `reverse_text(text)` - returns the text reversed
   - `capitalize_words(text)` - returns text with each word capitalized (first letter uppercase)
   - `remove_spaces(text)` - returns text with all spaces removed
2. **Main program:**
   - Ask the user to enter a sentence
   - Call each function with the user's input
   - Display all results in a formatted report
3. **Bonus:** Create a function `analyze_text(text)` that returns a dictionary with all statistics:

```
{
    "word_count": 5,
    "vowel_count": 8,
    "reversed": "...",
    "capitalized": "...",
    "no_spaces": "..."
}
```

**Example interaction:**

```
Enter a sentence: hello world python
========================================
Text Analysis:
========================================
Word count: 3
Vowel count: 4
Reversed: nohtyp dlrow olleh
Capitalized: Hello World Python
No spaces: helloworldpython
```

---

# Practice 3 – Student Management System (Functions)

Create `student_manager.py` that uses functions to manage student data:

1. **Create these functions:**
   - `add_student(students_dict, name, grades)` - adds a student with name and list of grades to dictionary
   - `calculate_average(grades)` - takes a list of grades and returns the average
   - `get_letter_grade(average)` - takes average and returns letter grade (A: 90+, B: 80-89, C: 70-79, D: 60-69, F: <60)
   - `find_top_student(students_dict)` - returns the name of student with highest average
   - `get_students_above_threshold(students_dict, threshold)` - returns list of student names with average above threshold
2. **Main program:**
   - Initialize an empty dictionary `students = {}`
   - Use a `while` loop with menu:
     - "1. Add student"
     - "2. View all students"
     - "3. Find top student"
     - "4. Find students above threshold"
     - "5. Exit"
   - For each menu option, call the appropriate functions

**Example interaction:**

```
Menu:
1. Add student
2. View all students
3. Find top student
4. Find students above threshold
5. Exit
Choice: 1
Student name: Alice
Enter grades (comma-separated): 85,90,88
Student added!

Choice: 2
All Students:
Alice: Average = 87.67, Grade = B

Choice: 3
Top student: Alice (Average: 87.67)

Choice: 4
Enter threshold: 85
Students above 85: ['Alice']
```

**Note:** For comma-separated grades input, you can use:

```python
grades_str = input("Enter grades (comma-separated): ")
grades = [int(x.strip()) for x in grades_str.split(",")]
```

---

# Practice 4 — Lambda Functions & Higher-Order Functions

Create `lambda_practice.py` that demonstrates lambda functions and higher-order operations:

1. **Create a list of numbers:**

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

2. **Using lambda functions, create:**
   - A lambda to square a number
   - A lambda to check if a number is even
   - A lambda to check if a number is greater than 5
3. **Using these lambdas with list comprehensions or loops:**
   - Create a list of squared numbers
   - Filter even numbers
   - Filter numbers greater than 5
   - Create a list of numbers that are both even AND greater than 5
4. **Create regular functions that do the same:**
   - `square_number(x)` - returns x squared
   - `is_even_num(x)` - returns True if x is even
   - `is_greater_than_five(x)` - returns True if x > 5
5. **Compare the results** - show that both lambda and regular functions produce the same results

**Example output:**

```
Original numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Using Lambda Functions:
Squared: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
Even numbers: [2, 4, 6, 8, 10]
Greater than 5: [6, 7, 8, 9, 10]
Even AND > 5: [6, 8, 10]

Using Regular Functions:
Squared: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
Even numbers: [2, 4, 6, 8, 10]
Greater than 5: [6, 7, 8, 9, 10]
Even AND > 5: [6, 8, 10]
```

---

# Practice 5 — Bank Account Class

Create `bank_account.py` with a `BankAccount` class:

1. **Class Requirements:**
   - `__init__(self, account_holder, initial_balance=0)` - initialize account with holder name and balance
   - `deposit(self, amount)` - adds amount to balance, returns new balance
   - `withdraw(self, amount)` - subtracts amount from balance if sufficient funds, returns new balance or None if insufficient
   - `get_balance(self)` - returns current balance
   - `display_info(self)` - prints formatted account information
2. **Main program:**
   - Create at least 2 `BankAccount` objects with different initial balances
   - Perform multiple deposits and withdrawals
   - Display account information after each operation
   - Show that each account maintains its own balance
3. **Bonus:** Add a class attribute `total_accounts` that counts how many accounts have been created

**Example interaction:**

```
Creating account for Alice with $100...
Account created!

Depositing $50 to Alice's account...
New balance: $150.00

Withdrawing $75 from Alice's account...
New balance: $75.00

Attempting to withdraw $100 from Alice's account...
Insufficient funds! Current balance: $75.00

Account Information:
====================
Account Holder: Alice
Balance: $75.00

Creating account for Bob with $200...
Account created!

Bob's balance: $200.00
Alice's balance: $75.00
(Each account is independent!)
```

---

# Mini Project — Library Management System

Create a project with the following structure:

```
library_system/
├── main.py
├── book.py
└── library.py
```

**Requirements:**

1. **In** book.py**, create a** Book **class:**
   - Attributes: title, author, isbn, available (boolean)
   - __init__(self, title, author, isbn) - initialize book
   - checkout(self) - marks book as unavailable if available, returns True/False
   - return_book(self) - marks book as available
   - get_info(self) - returns formatted string with book details
2. **In** library.py**, create a** Library **class:**
   - Attributes: books (list of Book objects), name
   - __init__(self, name) - initialize library with name
   - add_book(self, book) - adds a Book object to the library
   - find_book(self, title) - returns Book object if found, None otherwise
   - list_available_books(self) - returns list of available books
   - checkout_book(self, title) - finds book and checks it out, returns success message
   - return_book(self, title) - finds book and returns it, returns success message
   - display_all_books(self) - prints all books with their status
3. **In** main.py**:**
   - Create a Library instance
   - Create several Book instances
   - Add books to library
   - Implement a menu system:
     - "1. Add book"
     - "2. List all books"
     - "3. List available books"
     - "4. Checkout book"
     - "5. Return book"
     - "6. Exit"
   - Use functions to handle menu operations (e.g., handle_add_book(library), handle_checkout(library))

**Example interaction:**

```
Welcome to Python Library System!

Menu:
1. Add book
2. List all books
3. List available books
4. Checkout book
5. Return book
6. Exit

Choice: 1
Enter title: Python Basics
Enter author: John Doe
Enter ISBN: 123456
Book added!
```

# Functions & Modular Code Practice — Solutions

Python Course · Module 5 Practice Solutions

ElnurBDa

# Practice 1 — Function Library (Solution)

**math_utils.py:**

```python
def calculate_area(length, width):
    return length * width

def calculate_perimeter(length, width):
    return 2 * (length + width)

def is_even(number):
    return number % 2 == 0

def factorial(n):
    if n == 0 or n == 1:
        return 1
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

def power(base, exponent):
    result = 1
    for i in range(exponent):
        result *= base
    return result
```

**main.py:**

```python
from math_utils import (
    calculate_area,
    calculate_perimeter,
    is_even,
    factorial,
    power
)

# Get length and width
length = float(input("Enter length: "))
width = float(input("Enter width: "))

# Calculate area and perimeter
area = calculate_area(length, width)
perimeter = calculate_perimeter(length, width)

print(f"Area: {area}")
print(f"Perimeter: {perimeter}")

# Check if number is even
num = int(input("\nEnter a number: "))
print(f"Is even: {is_even(num)}")

# Calculate factorial
fact_num = int(input("\nEnter a number for factorial: "))
fact_result = factorial(fact_num)
print(f"Factorial of {fact_num}: {fact_result}")

# Calculate power
base = int(input("\nEnter base: "))
exponent = int(input("Enter exponent: "))
power_result = power(base, exponent)
print(f"{base}^{exponent} = {power_result}")
```

# Practice 2 – Text Processing Functions (Solution)

**Basic Functions** (Part 1):

```python
def count_words(text):
    words = []
    current_word = ""
    for char in text:
        if char == ' ':
            if current_word != "":
                words.append(current_word)
                current_word = ""
        else:
            current_word += char
    if current_word != "":
        words.append(current_word)
    return len(words)

def count_vowels(text):
    vowels = "aeiouAEIOU"
    count = 0
    for char in text:
        if char in vowels:
            count += 1
    return count

def reverse_text(text):
    reversed_str = ""
    for i in range(len(text) - 1, -1, -1):
        reversed_str += text[i]
    return reversed_str

def remove_spaces(text):
    result = ""
    for char in text:
        if char != ' ':
            result += char
    return result
```

**Basic Functions** (Part 2):

```python
def capitalize_words(text):
    # Split into words manually
    words = []
    current_word = ""
    for char in text:
        if char == ' ':
            if current_word != "":
                words.append(current_word)
                current_word = ""
        else:
            current_word += char
    if current_word != "":
        words.append(current_word)

    # Capitalize each word
    uppercase_map = {
        'a': 'A', 'b': 'B', 'c': 'C', 'd': 'D', 'e': 'E',
        'f': 'F', 'g': 'G', 'h': 'H', 'i': 'I', 'j': 'J',
        'k': 'K', 'l': 'L', 'm': 'M', 'n': 'N', 'o': 'O',
        'p': 'P', 'q': 'Q', 'r': 'R', 's': 'S', 't': 'T',
        'u': 'U', 'v': 'V', 'w': 'W', 'x': 'X', 'y': 'Y', 'z': 'Z'
    }

    capitalized_words = []
    for word in words:
        if len(word) > 0:
            first_char = word[0]
            if first_char in uppercase_map:
                first_char = uppercase_map[first_char]
            capitalized_word = first_char
            for i in range(1, len(word)):
                capitalized_word += word[i]
            capitalized_words.append(capitalized_word)

    # Join words with spaces manually
    result = ""
    for i, word in enumerate(capitalized_words):
        if i > 0:
            result += " "
        result += word

    return result
```

# Practice 2 — Text Processing Functions (Solution)

**Main Program:**

```python
from text_processor import (
    count_words,
    count_vowels,
    reverse_text,
    capitalize_words,
    remove_spaces
)

# Get user input
sentence = input("Enter a sentence: ")

# Display individual results
print("=" * 40)
print("Text Analysis:")
print("=" * 40)
print(f"Word count: {count_words(sentence)}")
print(f"Vowel count: {count_vowels(sentence)}")
print(f"Reversed: {reverse_text(sentence)}")
print(f"Capitalized: {capitalize_words(sentence)}")
print(f"No spaces: {remove_spaces(sentence)}")
```

**Bonus Function:**

```python
def analyze_text(text):
    return {
        "word_count": count_words(text),
        "vowel_count": count_vowels(text),
        "reversed": reverse_text(text),
        "capitalized": capitalize_words(text),
        "no_spaces": remove_spaces(text)
    }

# Using analyze_text function
print("\n" + "=" * 40)
print("Using analyze_text function:")
print("=" * 40)
analysis = analyze_text(sentence)
for key, value in analysis.items():
    print(f"{key}: {value}")
```

---

# Practice 3 — Student Management System (Solution)

**Functions:**

```python
def add_student(students_dict, name, grades):
    students_dict[name] = grades

def calculate_average(grades):
    total = 0
    for grade in grades:
        total += grade
    return total / len(grades)

def get_letter_grade(average):
    if average >= 90:
        return "A"
    elif average >= 80:
        return "B"
    elif average >= 70:
        return "C"
    elif average >= 60:
        return "D"
    else:
        return "F"

def find_top_student(students_dict):
    top_name = None
    top_average = -1

    for name, grades in students_dict.items():
        avg = calculate_average(grades)
        if avg > top_average:
            top_average = avg
            top_name = name

    return top_name, top_average

def get_students_above_threshold(students_dict, threshold):
    above_threshold = []
    for name, grades in students_dict.items():
        avg = calculate_average(grades)
        if avg > threshold:
            above_threshold.append(name)
    return above_threshold
```

**Main Program:**

```python
from student_functions import (
    add_student,
    calculate_average,
    get_letter_grade,
    find_top_student,
    get_students_above_threshold
)

students = {}

while True:
    print("\nMenu:")
    print("1. Add student")
    print("2. View all students")
    print("3. Find top student")
    print("4. Find students above threshold")
    print("5. Exit")

    choice = input("Choice: ")

    if choice == "1":
        name = input("Student name: ")
        grades_str = input("Enter grades (comma-separated): ")
        grades = [int(x.strip()) for x in grades_str.split(",")]
        add_student(students, name, grades)
        print("Student added!")

    elif choice == "2":
        print("\nAll Students:")
        for name, grades in students.items():
            avg = calculate_average(grades)
            letter = get_letter_grade(avg)
            print(f"{name}: Average = {avg:.2f}, Grade = {letter}")

    elif choice == "3":
        if len(students) == 0:
            print("No students found!")
        else:
            top_name, top_avg = find_top_student(students)
            print(f"Top student: {top_name} (Average: {top_avg:.2f})")

    elif choice == "4":
        threshold = float(input("Enter threshold: "))
        above = get_students_above_threshold(students, threshold)
        print(f"Students above {threshold}: {above}")

    elif choice == "5":
        print("Goodbye!")
        break

    else:
        print("Invalid choice!")
```

# Practice 4 — Lambda Functions & Higher-Order Functions (Solution)

**Solution**:

```python
# Original list
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

print(f"Original numbers: {numbers}\n")

# Lambda functions
square_lambda = lambda x: x * x
is_even_lambda = lambda x: x % 2 == 0
is_greater_than_five_lambda = lambda x: x > 5

# Using lambdas with list comprehensions
squared_lambda = [square_lambda(x) for x in numbers]
even_lambda = [x for x in numbers if is_even_lambda(x)]
greater_five_lambda = [x for x in numbers if is_greater_than_five_lambda(x)]
even_and_greater_lambda = [
    x for x in numbers
    if is_even_lambda(x) and is_greater_than_five_lambda(x)
]

print("Using Lambda Functions:")
print(f"Squared: {squared_lambda}")
print(f"Even numbers: {even_lambda}")
print(f"Greater than 5: {greater_five_lambda}")
print(f"Even AND > 5: {even_and_greater_lambda}\n")

# Regular functions
def square_number(x):
    return x * x

def is_even_num(x):
    return x % 2 == 0

def is_greater_than_five(x):
    return x > 5

# Using regular functions with list comprehensions
squared_func = [square_number(x) for x in numbers]
even_func = [x for x in numbers if is_even_num(x)]
greater_five_func = [x for x in numbers if is_greater_than_five(x)]
even_and_greater_func = [
    x for x in numbers
    if is_even_num(x) and is_greater_than_five(x)
]

print("Using Regular Functions:")
print(f"Squared: {squared_func}")
print(f"Even numbers: {even_func}")
print(f"Greater than 5: {greater_five_func}")
print(f"Even AND > 5: {even_and_greater_func}\n")

# Verify they produce same results
print("Verification:")
print(f"Lambdas and functions produce same results: {squared_lambda ==
squared_func}")
```

**Alternative with map()** (if you want to show higher-order functions):

```python
# Original list
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Using map() with lambda
square_lambda = lambda x: x * x
squared_map = list(map(square_lambda, numbers))

# Using filter() with lambda
is_even_lambda = lambda x: x % 2 == 0
even_filter = list(filter(is_even_lambda, numbers))

# Combining filter conditions
is_greater_than_five = lambda x: x > 5
even_and_greater = list(filter(
    lambda x: is_even_lambda(x) and is_greater_than_five(x),
    numbers
))

print(f"Original: {numbers}")
print(f"Squared (map): {squared_map}")
print(f"Even (filter): {even_filter}")
print(f"Even AND > 5: {even_and_greater}")
```

# Practice 5 – Bank Account Class (Solution)

**BankAccount Class:**

```python
class BankAccount:
    total_accounts = 0  # Class attribute

    def __init__(self, account_holder, initial_balance=0):
        self.account_holder = account_holder
        self.balance = initial_balance
        BankAccount.total_accounts += 1
        print(f"Account created for {account_holder} with ${initial_balance
:.2f}!")

    def deposit(self, amount):
        self.balance += amount
        return self.balance

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            return self.balance
        else:
            return None

    def get_balance(self):
        return self.balance

    def display_info(self):
        print("\nAccount Information:")
        print("=" * 20)
        print(f"Account Holder: {self.account_holder}")
        print(f"Balance: ${self.balance:.2f}")
```

**Main Program:**

```python
from bank_account import BankAccount

# Create accounts
print("Creating account for Alice with $100...")
alice_account = BankAccount("Alice", 100)

print("\nDepositing $50 to Alice's account...")
new_balance = alice_account.deposit(50)
print(f"New balance: ${new_balance:.2f}")

print("\nWithdrawing $75 from Alice's account...")
new_balance = alice_account.withdraw(75)
if new_balance is not None:
    print(f"New balance: ${new_balance:.2f}")
else:
    print("Insufficient funds!")

print("\nAttempting to withdraw $100 from Alice's account...")
result = alice_account.withdraw(100)
if result is None:
    print(f"Insufficient funds! Current balance: ${alice_account.get_balance
():.2f}")

alice_account.display_info()

# Create second account
print("\n" + "=" * 40)
print("Creating account for Bob with $200...")
bob_account = BankAccount("Bob", 200)

print(f"\nBob's balance: ${bob_account.get_balance():.2f}")
print(f"Alice's balance: ${alice_account.get_balance():.2f}")
print("(Each account is independent!)")

# Display total accounts
print(f"\nTotal accounts created: {BankAccount.total_accounts}")
```

# Mini Project – Library Management System (Solution)

**book.py:**

```
class Book:
    def __init__(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn
        self.available = True

    def checkout(self):
        if self.available:
            self.available = False
            return True
        return False

    def return_book(self):
        self.available = True

    def get_info(self):
        status = "Available" if self.available
else "Checked Out"
        return f"{self.title} by {self.author}
(ISBN: {self.isbn}) - {status}"
```

**library.py:**

```
from book import Book

class Library:
    def __init__(self, name):
        self.name = name
        self.books = []

    def add_book(self, book):
        self.books.append(book)

    def find_book(self, title):
        for book in self.books:
            if book.title.lower() == title.
lower():
                return book
        return None

    def list_available_books(self):
        available = []
        for book in self.books:
            if book.available:
                available.append(book)
        return available

    def checkout_book(self, title):
        book = self.find_book(title)
        if book is None:
            return "Book not found!"
        if book.checkout():
            return "Book checked out
successfully!"
        return "Book is already checked out!"

    def return_book(self, title):
        book = self.find_book(title)
        if book is None:
            return "Book not found!"
        book.return_book()
        return "Book returned successfully!"

    def display_all_books(self):
        if len(self.books) == 0:
            print("No books in library.")
        else:
            for i, book in enumerate(self
.books, 1):
                print(f"{i}. {book.get_info
()}")
```

**main.py:**

```
from library import Library
from book import Book

def handle_add_book(library):
    title = input("Enter title: ")
    author = input("Enter author: ")
    isbn = input("Enter ISBN: ")
    book = Book(title, author, isbn)
    library.add_book(book)
    print("Book added!")

def handle_list_all(library):
    print("\nAll Books:")
    library.display_all_books()

def handle_list_available(library):
    available = library.list_available_books()
    if len(available) == 0:
        print("No available books.")
    else:
        print("\nAvailable Books:")
        for i, book in enumerate(available, 1):
            print(f"{i}. {book.get_info()}")

def handle_checkout(library):
    title = input("Enter book title: ")
    message = library.checkout_book(title)
    print(message)

def handle_return(library):
    title = input("Enter book title: ")
    message = library.return_book(title)
    print(message)

# Main program
library = Library("Python Library System")

# Add some initial books
book1 = Book("Python Basics", "John Doe", "
123456")
book2 = Book("Advanced Python", "Jane Smith", "
789012")
library.add_book(book1)
library.add_book(book2)

print("Welcome to Python Library System!\n")

while True:
    print("Menu:")
    print("1. Add book")
    print("2. List all books")
    print("3. List available books")
    print("4. Checkout book")
    print("5. Return book")
```

# Files & Error Handling

Python Course · Module 6

ElnurBDa

# Module 6 — Files & Error Handling

In this module we learn how to **read and write files** and **handle errors gracefully**.

We will learn:

- Reading & writing files
- Handling errors with `try` / `except`
- Working with CSV files
- Best practices for error handling

---

# Files & Persistent Storage

Programs normally store data in **RAM** (temporary).
Files store data on **disk** (persistent).

Why use files?

- Save logs
- Export reports
- Config files
- Simple data exchange

---

# File Paths & Modes

To work with a file, Python needs:

- **Path** (where it is)
- **Mode** (what we want to do)

Common modes:

```
"r"   read (error if missing)
"w"   write, truncate file
"a"   append to end
"r+"  read & write
"rb"  read binary
"wb"  write binary
```

Always prefer **absolute paths** in bigger projects.

---

# Opening Files Safely — with

Use with to open files — it auto-closes them.

```
# reading a text file
path = "notes.txt"

with open(path, "r", encoding="utf-8") as f:
    content = f.read()
    print("File content:")
    print(content)
```

[finished]

```
File content:
some test text is here
yay!
```

Benefits:

- No need to call close()
- Works well with exceptions

# Reading Files — Variants

```python
with open("notes.txt", "r", encoding="utf-8") as f:
    all_text = f.read()        # whole file
    print("LEN:", len(all_text))
```

[finished]

```
LEN: 28
```

```python
with open("notes.txt", "r", encoding="utf-8") as f:
    first_line = f.readline()  # one line
    print("FIRST:", first_line)
```

[finished]

```
FIRST: some test text is here
```

```python
with open("notes.txt", "r", encoding="utf-8") as f:
    lines = f.readlines()      # list of lines
    print("COUNT:", len(lines))
```

[finished]

```
COUNT: 2
```

Choose based on file size and use-case.

# Writing & Appending

```python
# overwrite or create
with open("log.txt", "w", encoding="utf-8") as f:
    f.write("First run\n")
    f.write("Program started successfully.\n")
```

——————————— [finished] ———————————

```python
# append
with open("log.txt", "a", encoding="utf-8") as f:
    f.write("Another run...\n")
```

——————————— [finished] ———————————

Notes:

- "w" **deletes** old content
- "a" keeps content and adds at the end

---

# Working with CSV Files

CSV (Comma-Separated Values) is a simple text format for tables.

```python
import csv

rows = [
    ["id", "username", "score"],
    [1, "admin", 99],
    [2, "guest", 42],
]

with open("users.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerows(rows)

with open("users.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

[finished]

```
['id', 'username', 'score']
['1', 'admin', '99']
['2', 'guest', '42']
```

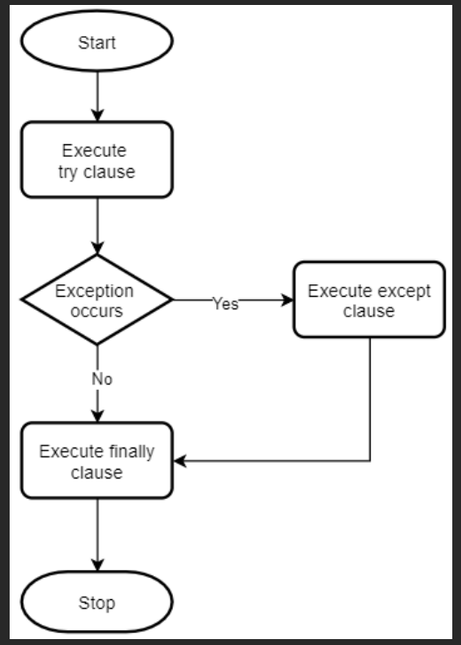
# Exceptions — What & Why

An **exception** is a runtime error that stops normal execution.

Common sources:

- Invalid input
- Missing files
- Network errors
- Database problems

Goal: **fail safely**, show useful messages, keep program controllable.



# Basic try / except

```python
# text = input("Enter a number: ")
text = "Salam"

try:
    n = int(text)
    print("Squared:", n * n)
except ValueError:
    print("That was not a valid integer!")
```

[finished]

```
That was not a valid integer!
```

Flow:

1. Run code in try block
2. If exception matches except, handle it
3. Program continues instead of crashing

# Error Handling Patterns in Bigger Scripts

As programs grow, we want **clear places** where errors are handled.

Typical pattern:

```
def run():
    try:
        # 1) read config
        # 2) open files / database
        # 3) main logic
        ...
    except FileNotFoundError as e:
        print("Missing file:", e)
    except ValueError as e:
        print("Bad data:", e)
```

Benefits:

- All top-level errors are handled in **one place**
- Internal functions can either handle or **raise** exceptions

In real projects we often move these messages to **logging** instead of `print()`.

---

# 0_o

```python
# text = input("Enter a number: ")
text = "3"

try:
    n = int(text)
    print("Squared:", n * n)
except ValueError:
    print("That was not a valid integer!")
```

[finished]

```
Squared: 9
```

# Catching Multiple Exceptions

```python
def read_number_from_file(path):
    try:
        with open(path, "r", encoding="utf-8") as f:
            text = f.read().strip()
            return int(text)
    except FileNotFoundError:
        print("File does not exist:", path)
    except ValueError:
        print("File did not contain a valid number.")
```

You can also group them:

```python
except (FileNotFoundError, PermissionError) as e:
    print("File access problem:", e)
```

---

# else and finally

```python
path = "config.txt"

try:
    f = open(path, "r", encoding="utf-8")
except FileNotFoundError:
    print("Config missing")
else:
    print("Config loaded:", f.readline().strip())
    f.close()
finally:
    print("This always runs (cleanup, logs, etc.)")
```

[finished]

```
Config missing
This always runs (cleanup, logs, etc.)
```

- `else`: runs if **no** exception happened
- `finally`: runs **always**, used for cleanup

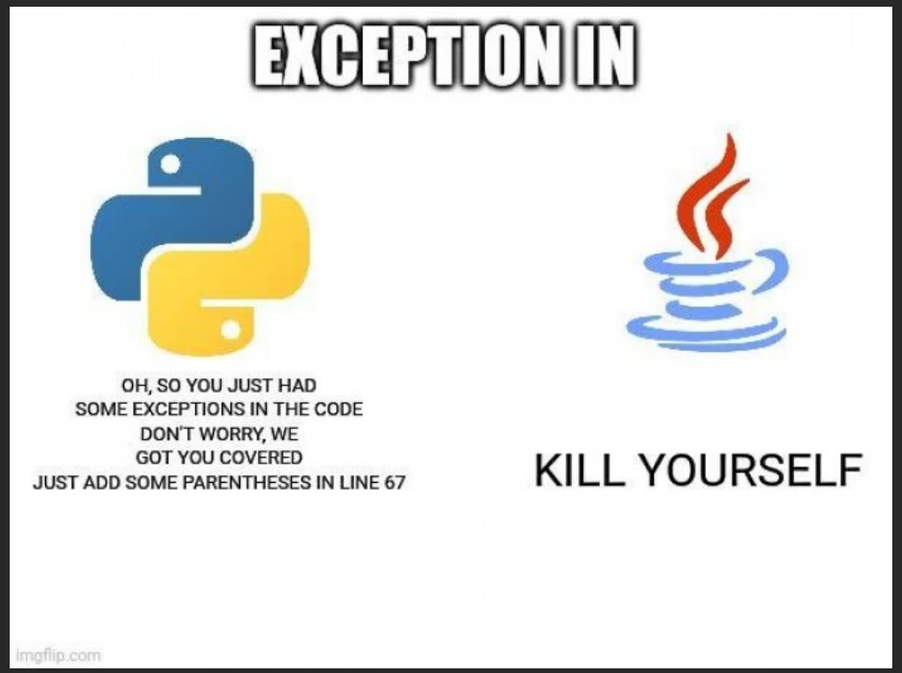
# Mini Task

Create a **log tracker** program:

1. Ask user for their name and action
2. Write to activity.log with timestamp
3. Handle file errors gracefully
4. Read and display all logs
5. Add option to clear logs (with confirmation)

Bonus:

- Save logs as CSV with csv module
- Add logging levels (INFO, ERROR, WARNING)
- Filter logs by action or date



# Thanks!

By ElnurBDa

# Files & Error Handling Practice

Python Course · Module 6 Practice

ElnurBDa

# Practice 1 — Robust Score Importer

Create `score_importer.py` that reads a text file with one student score per line, calculates simple statistics, and handles bad data safely.

**File format (example `scores.txt`):**

```
95
80
abc
102
67
```

**Requirements:**

1. Ask the user for the **absolute path** to a scores file.
2. Try to open the file for reading.
   - If the file does not exist or cannot be opened, print a clear message and **exit** gracefully.
3. Read the file **line by line**:
   - Strip whitespace from each line
   - Use `try` / `except` to convert each line to `int`
   - If a line is not a valid integer, **skip it** and print a warning with the line content
   - Ignore scores that are **less than 0 or greater than 100**, with a warning
4. Store all valid scores in a **list**.
5. If there are no valid scores, print a message and exit.
6. Otherwise compute:
   - count of valid scores
   - minimum, maximum
   - average (float, 2 decimal places)
7. Print a formatted summary.

**Bonus:**

- Use a **function** like `load_scores(path)` that returns the list of valid scores.
- Return `None` or an empty list when something goes wrong and handle it in `main`.

---

# Practice 2 — Simple User Database (CSV)

Create `user_database.py` that manages a **CSV file of users** with safe file handling.

**CSV structure (`users.csv`):**

```
id,username,email,age
1,alice,alice@example.com,25
2,bob,bob@example.com,19
```

**Requirements:**

1. Use the built-in `csv` module.
2. At program start:
   - Try to open the CSV file (ask user for **absolute path** or use a default path)
   - If the file does not exist:
     - Create it and write a **header row**
     - Start with an empty list of users
3. Represent each user as a **dictionary**:

```
{
    "id": int,
    "username": str,
    "email": str,
    "age": int
}
```

4. Implement a menu in a `while True` loop:
   - `1. Add user`
   - `2. List users`
   - `3. Find users by minimum age`
   - `4. Save & Exit`
5. For **Add user**:
   - Ask for username, email, age
   - Validate age (`int`, `>= 0`) with `try` / `except`
   - Assign a new `id` (max existing id + 1, or 1 if empty)
   - Append to the in-memory list
6. For **List users**:
   - Print all users in a neat table-like format.
7. For **Find users by minimum age**:
   - Ask the user for a minimum age (int, with validation)
   - Use a **list comprehension** or loop to filter users
8. On **Save & Exit**:
   - Write all users back to the CSV file
   - Use `try` / `except` to handle file write errors and print a clear message.

**Bonus:**

- Wrap file-related code in helper functions (`load_users`, `save_users`).
- Use `else` / `finally` where it makes sense.

---

# Practice 3 — Log Analyzer with Error Handling

Create `log_analyzer.py` that reads a log file,
extracts **basic statistics**, and survives bad input or missing files.

**Example log file (`app.log`):**

```
2025-12-01 10:00:00 INFO User alice logged in
2025-12-01 10:05:23 ERROR Failed to load profile
2025-12-01 10:06:10 WARNING Slow response
2025-12-01 10:10:45 INFO User bob logged out
```

**Requirements:**

1. Ask the user for the **absolute path** to the log file.
2. Try to open and read the file:
   - If it fails, print a friendly error and exit.
3. For each line, try to parse it into parts:
   - date (string up to first space)
   - time (string between first and second space)
   - level (`INFO`, `ERROR`, `WARNING`, …)
   - message (rest of the line)
4. Use `split()` and **defensive programming**:
   - If a line does not have at least 4 parts, skip it with a warning.
5. Use a **dictionary** to count how many messages for each level:

```
counts = {"INFO": 0, "ERROR": 0, "WARNING": 0, ...}
```

6. Store all **ERROR** messages in a **list** of dictionaries:

```
{
    "datetime": "2025-12-01 10:05:23",
    "message": "Failed to load profile"
}
```

7. After processing:
   - Print total lines processed
   - Print count per log level
   - Print all ERROR entries in a readable format
8. Use at least one helper **function**, e.g. `parse_log_line(line)` that returns either a dict or `None`.

**Bonus:**

- Add a menu option to **filter by date** (e.g. show only entries that start with a given date string).
- Handle user input errors with `try` / `except`.

---

# Practice 4 — Task Manager with File Persistence & Exceptions

Extend the earlier **task manager** idea into `task_manager_files.py` that stores tasks in a JSON-like text format (you can invent a simple format) and uses error handling.

**Requirements:**

1. Each task is represented as a **dictionary**:

```
{
    "id": int,
    "title": str,
    "priority": str,  # "high", "medium", "low"
    "completed": bool
}
```

2. At program start:
   - Ask for an **absolute path** to a task file (e.g. `tasks.txt`)
   - Try to open and read it
   - If file does not exist:
     - Start with an **empty list** of tasks
     - Create the file when saving
3. Choose a **simple line-based format**, for example:

```
1|Learn Python|high|False
2|Exercise|medium|True
```

4. Implement helper functions:
   - `load_tasks(path)` → returns list of task dictionaries (with `try` / `except`)
   - `save_tasks(path, tasks)` → writes tasks to file safely
5. Implement a `while` loop menu:
   - `1. Add task`
   - `2. Mark task as complete`
   - `3. List tasks`
   - `4. List incomplete tasks`
   - `5. Delete task`
   - `6. Save & Exit`
6. Use **safe integer parsing** with `try` / `except` for IDs and choices.
7. When loading tasks:
   - Use `try` / `except` when splitting/parsing lines
   - Skip malformed lines with a warning instead of crashing
8. When saving tasks:
   - Catch file write errors and show a clear message.

**Bonus:**

- Add a small `Task` class with methods like `mark_done()` and `to_line()` / `from_line()`.
- Use `finally` to print a goodbye or cleanup message no matter how the program exits.

---

# Files & Error Handling Practice — Solutions

Python Course · Module 6 Practice Solutions

ElnurBDa

# Solutions Overview

These solutions are **one possible implementation** for each practice task.
You can (and should!) experiment with different designs as long as:

- Files are handled safely
- Exceptions are caught where appropriate
- Data structures and control flow are used clearly

---

# Practice 1 — Robust Score Importer (Solution)

**score_importer.py** — core logic:

```python
def load_scores(path):
    scores = []

    try:
        with open(path, "r", encoding="utf-8") as f:
            for line_number, line in enumerate(f, start=1):
                text = line.strip()
                if text == "":
                    continue

                try:
                    value = int(text)
                except ValueError:
                    print(f"[WARN] Line {line_number}: not a valid integer
-> {text!r}")
                    continue

                if value < 0 or value > 100:
                    print(f"[WARN] Line {line_number}: score out of range
(0-100) -> {value}")
                    continue

                scores.append(value)

    except FileNotFoundError:
        print("File not found:", path)
        return []
    except OSError as e:
        print("Could not open file:", e)
        return []

    return scores
```

**Main program:**

```python
def main():
    path = input("Enter absolute path to scores file: ").strip()
    scores = load_scores(path)

    if not scores:
        print("No valid scores found. Exiting.")
        return

    count = len(scores)
    minimum = min(scores)
    maximum = max(scores)
    average = sum(scores) / count

    print("\nScore Summary")
    print("=============")
    print(f"Valid scores: {count}")
    print(f"Min: {minimum}")
    print(f"Max: {maximum}")
    print(f"Average: {average:.2f}")


if __name__ == "__main__":
    main()
```

# Practice 2 — Simple User Database (CSV) (Solution)

**Helper functions (user_database.py):**

```python
import csv


def load_users(path):
    users = []
    try:
        with open(path, "r", encoding="utf-8") as f:
            reader = csv.DictReader(f)
            for row in reader:
                try:
                    user = {
                        "id": int(row["id"]),
                        "username": row["username"],
                        "email": row["email"],
                        "age": int(row["age"]),
                    }
                    users.append(user)
                except (KeyError, ValueError):
                    print("[WARN] Skipping malformed row:", row)
    except FileNotFoundError:
        print("CSV file not found, creating a new one at save time.")
    except OSError as e:
        print("Error reading CSV file:", e)
    return users


def save_users(path, users):
    fieldnames = ["id", "username", "email", "age"]
    try:
        with open(path, "w", newline="", encoding="utf-8") as f:
            writer = csv.DictWriter(f, fieldnames=fieldnames)
            writer.writeheader()
            for user in users:
                writer.writerow(user)
        print("Users saved successfully.")
    except OSError as e:
        print("Error writing CSV file:", e)
```

**Menu logic:**

```python
def get_next_id(users):
    if not users:
        return 1
    return max(user["id"] for user in users) + 1


def add_user(users):
    username = input("Username: ").strip()
    email = input("Email: ").strip()

    try:
        age = int(input("Age: ").strip())
        if age < 0:
            print("Age cannot be negative.")
            return
    except ValueError:
        print("Invalid age. Must be an integer.")
        return

    new_id = get_next_id(users)
    users.append({"id": new_id, "username": username, "email": email, "age":
age})
    print("User added!")
```

# Practice 2 — Simple User Database (CSV) (Solution)

**More menu actions:**

```python
def list_users(users):
    if not users:
        print("No users in database.")
        return

    print("\nUsers")
    print("=====")
    for user in users:
        print(
            f"{user['id']:>3}  "
            f"{user['username']:<15}  "
            f"{user['email']:<25}  "
            f"age={user['age']}"
        )


def find_by_min_age(users):
    try:
        min_age = int(input("Minimum age: ").strip())
    except ValueError:
        print("Invalid age.")
        return

    filtered = [u for u in users if u["age"] >= min_age]

    if not filtered:
        print("No users with that minimum age.")
        return

    print(f"\nUsers with age >= {min_age}")
    print("========================")
    for user in filtered:
        print(f"{user['id']} - {user['username']} ({user['age']} years)")
```

**Main loop:**

```python
def main():
    path = input("Enter CSV path (absolute): ").strip()
    users = load_users(path)

    while True:
        print("\nMenu:")
        print("1. Add user")
        print("2. List users")
        print("3. Find users by minimum age")
        print("4. Save & Exit")

        choice = input("Choice: ").strip()

        if choice == "1":
            add_user(users)
        elif choice == "2":
            list_users(users)
        elif choice == "3":
            find_by_min_age(users)
        elif choice == "4":
            save_users(path, users)
            break
        else:
            print("Invalid choice.")


if __name__ == "__main__":
    main()
```

# Practice 3 — Log Analyzer with Error Handling (Solution)

**Parser helper:**

```python
def parse_log_line(line):
    line = line.strip()
    if not line:
        return None

    parts = line.split(" ", 3)  # max 4 parts
    if len(parts) < 4:
        print("[WARN] Malformed log line (too few parts):", line)
        return None

    date_part, time_part, level_part, message_part = parts
    entry = {
        "datetime": f"{date_part} {time_part}",
        "level": level_part,
        "message": message_part,
    }
    return entry
```

**Analyzer logic:**

```python
def analyze_log(path):
    entries = []
    try:
        with open(path, "r", encoding="utf-8") as f:
            for line in f:
                entry = parse_log_line(line)
                if entry is not None:
                    entries.append(entry)
    except FileNotFoundError:
        print("Log file not found:", path)
        return []
    except OSError as e:
        print("Error reading log file:", e)
        return []

    return entries
```

# Practice 3 — Log Analyzer with Error Handling (Solution)

**Counting and reporting:**

```python
def build_level_counts(entries):
    counts = {}
    for entry in entries:
        level = entry["level"]
        if level not in counts:
            counts[level] = 0
        counts[level] += 1
    return counts


def print_report(entries):
    print("\nLog Report")
    print("==========")
    print("Total parsed entries:", len(entries))

    counts = build_level_counts(entries)
    print("\nBy level:")
    for level, count in counts.items():
        print(f"- {level}: {count}")

    error_entries = [
        e for e in entries if e["level"].upper() == "ERROR"
    ]

    if error_entries:
        print("\nERROR entries:")
        for e in error_entries:
            print(f"[{e['datetime']}] {e['message']}")
    else:
        print("\nNo ERROR entries found.")
```

**Main function:**

```python
def main():
    path = input("Enter absolute path to log file: ").strip()
    entries = analyze_log(path)
    if not entries:
        print("No entries to report.")
        return

    print_report(entries)


if __name__ == "__main__":
    main()
```

# Practice 4 — Task Manager with File Persistence (Solution)

**Line format helpers:**

```
def task_to_line(task):
    return f"{task['id']}|{task['title']}|{task['priority']}|{task['
completed']}"


def line_to_task(line):
    parts = line.strip().split("|")
    if len(parts) != 4:
        raise ValueError("Wrong number of parts")

    task_id_str, title, priority, completed_str = parts

    task_id = int(task_id_str)
    completed = completed_str == "True"

    return {
        "id": task_id,
        "title": title,
        "priority": priority,
        "completed": completed,
    }
```

**File helpers with error handling:**

```
def load_tasks(path):
    tasks = []
    try:
        with open(path, "r", encoding="utf-8") as f:
            for line_number, line in enumerate(f, start=1):
                line = line.strip()
                if not line:
                    continue
                try:
                    task = line_to_task(line)
                    tasks.append(task)
                except (ValueError, TypeError) as e:
                    print(f"[WARN] Skipping malformed task at line
{line_number}: {e}")
    except FileNotFoundError:
        print("Task file not found, starting with empty task list.")
    except OSError as e:
        print("Error reading tasks file:", e)

    return tasks
```

# Practice 4 — Task Manager with File Persistence (Solution)

**Saving tasks:**

```
def save_tasks(path, tasks):
    try:
        with open(path, "w", encoding="utf-8") as f:
            for task in tasks:
                f.write(task_to_line(task) + "\n")
        print("Tasks saved successfully.")
    except OSError as e:
        print("Error writing tasks file:", e)
```

**Utility helpers:**

```
def next_task_id(tasks):
    if not tasks:
        return 1
    return max(t["id"] for t in tasks) + 1
```

**Menu actions:**

```
def add_task(tasks):
    title = input("Task title: ").strip()
    priority = input("Priority (high/medium/low): ").strip().lower()
    if priority not in ("high", "medium", "low"):
        print("Invalid priority, defaulting to 'medium'.")
        priority = "medium"
    task_id = next_task_id(tasks)
    tasks.append(
        {"id": task_id, "title": title, "priority": priority, "completed":
False}
    )
    print("Task added!")
```

# Practice 4 — Task Manager with File Persistence (Solution)

**More menu actions:**

```python
def mark_complete(tasks):
    try:
        task_id = int(input("Enter task ID to mark complete: ").strip())
    except ValueError:
        print("Invalid ID.")
        return

    for task in tasks:
        if task["id"] == task_id:
            task["completed"] = True
            print("Task marked as complete.")
            return

    print("Task not found.")


def list_tasks(tasks, only_incomplete=False):
    if not tasks:
        print("No tasks.")
        return

    print("\nTasks")
    print("=====")
    for task in tasks:
        if only_incomplete and task["completed"]:
            continue
        status = "✓" if task["completed"] else "✗"
        print(
            f"{task['id']:>3}  [{status}]  "
            f"{task['priority']:<6}  {task['title']}"
        )
```

**Delete and main loop:**

```python
def delete_task(tasks):
    try:
        task_id = int(input("Enter task ID to delete: ").strip())
    except ValueError:
        print("Invalid ID.")
        return

    for i, task in enumerate(tasks):
        if task["id"] == task_id:
            tasks.pop(i)
            print("Task deleted.")
            return

    print("Task not found.")
```

```python
def main():
    path = input("Enter absolute path to task file: ").strip()
    tasks = load_tasks(path)

    try:
        while True:
            print("\nMenu:")
            print("1. Add task")
            print("2. Mark task as complete")
            print("3. List tasks")
            print("4. List incomplete tasks")
            print("5. Delete task")
            print("6. Save & Exit")

            choice = input("Choice: ").strip()

            if choice == "1":
                add_task(tasks)
            elif choice == "2":
                mark_complete(tasks)
            elif choice == "3":
                list_tasks(tasks, only_incomplete=False)
            elif choice == "4":
                list_tasks(tasks, only_incomplete=True)
            elif choice == "5":
                delete_task(tasks)
            elif choice == "6":
                save_tasks(path, tasks)
                break
            else:
                print("Invalid choice.")
    finally:
        print("Goodbye!")


if __name__ == "__main__":
    main()
```

# Version Control with Git

Python Course · Git Module

ElnurBDa

# Module — Version Control with Git

In this module we learn how to **track changes** in our code and **collaborate** with others:

- What is version control and why we need it
- History of Git and its creator
- Git basics and core concepts
- Essential Git commands (add, commit, pull, push)
- What is GitHub and how to use it
- Practical example: pushing a Python app to GitHub

Goal: make your code **safe**, **trackable**, and **shareable**.

---

# The Problem: Code Without Version Control

Imagine working on a project without Git:

```
app.py
app_backup.py
app_final.py
app_final_v2.py
app_final_v2_REALLY_FINAL.py
app_old.py
app_working.py
```

Problems:

- **Lost changes** — accidentally deleted code
- **No history** — can't see what changed or why
- **Collaboration chaos** — multiple people editing the same file
- **No rollback** — can't undo mistakes easily

Version control solves all of these problems.

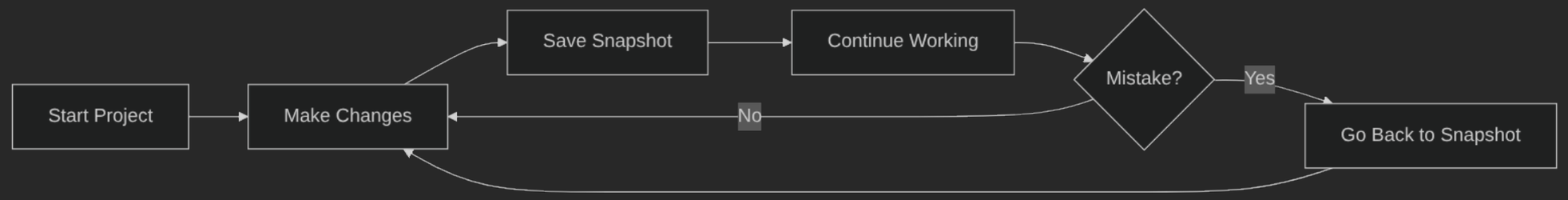
---

# What is Version Control?

**Version control** is a system that tracks changes to files over time.

Think of it like:

- **Time machine** for your code
- **Undo button** that works across days/weeks
- **Collaboration tool** for teams
- **Backup system** that keeps history




Every "snapshot" (commit) saves the entire state of your project at that moment.

---

# Why We Need Version Control

**For Individuals:**

- Track what changed and when
- Revert to previous working versions
- Experiment safely (branches)
- Understand your own code evolution

**For Professionals:**

- Required in almost all software jobs
- Essential for DevOps workflows
- Industry standard for code management
- Foundation for CI/CD pipelines

**For Teams:**

- Multiple people work on same codebase
- Merge changes automatically
- See who changed what
- Resolve conflicts systematically

---

# The History of Git

Git was created by **Linus Torvalds** in 2005.

**Context:**

- Linux kernel development needed better version control
- Previous tools were slow, proprietary, or expensive
- Needed something **fast**, **distributed**, and **free**

**Design Philosophy:**

- **Distributed** — every copy is a full repository
- **Fast** — optimized for speed
- **Simple** — core concepts are straightforward
- **Open source** — free for everyone

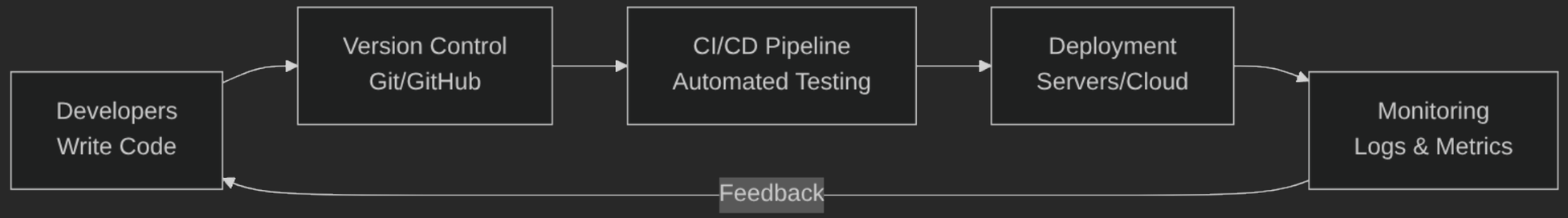
Git became the **de facto standard** for version control in software development.

---

# What is DevOps?

**DevOps** = **Dev**elopment + **Op**erations

It's a **culture and set of practices** that bring together software development and IT operations.




**Key Principles:**

- **Automation** — reduce manual work
- **Collaboration** — devs and ops work together
- **Continuous Integration** — merge code frequently
- **Continuous Deployment** — release often and safely

---

# Why Git is DevOps

Git is a **foundational DevOps tool** because:

**1. Enables Collaboration**

- Multiple developers work on the same codebase
- Changes are tracked and merged systematically

**2. Supports Automation**

- CI/CD pipelines trigger on Git events (push, pull request)
- Automated tests run when code changes

**3. Provides Traceability**

- Every change is logged with author, date, and message
- Essential for debugging and compliance

**4. Facilitates Deployment**

- Different branches for dev, staging, production
- Rollback to previous versions when needed

**5. Integrates with Tools**

- GitHub, GitLab, Bitbucket (hosting)
- Jenkins, GitHub Actions (automation)
- Docker, Kubernetes (deployment)

---

# Git Basics — Repository

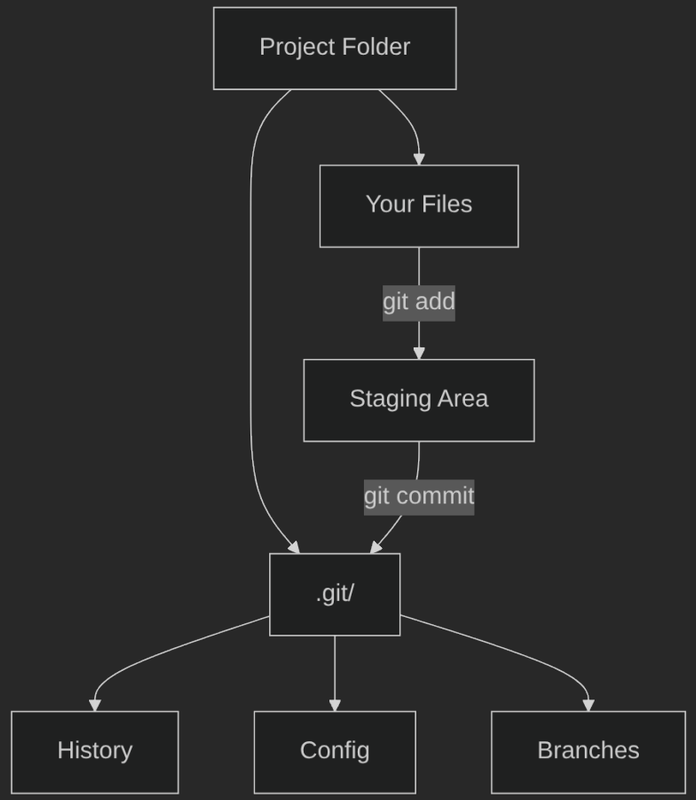
A **repository** (repo) is a folder that Git tracks.

Initialize a repository:

```
cd my_project
git init
```

This creates a hidden .git folder that stores:

- All file versions (history)
- Commit messages
- Branch information
- Configuration



# Git Basics — Three States

Files in Git can be in three states:

**1. Working Directory**

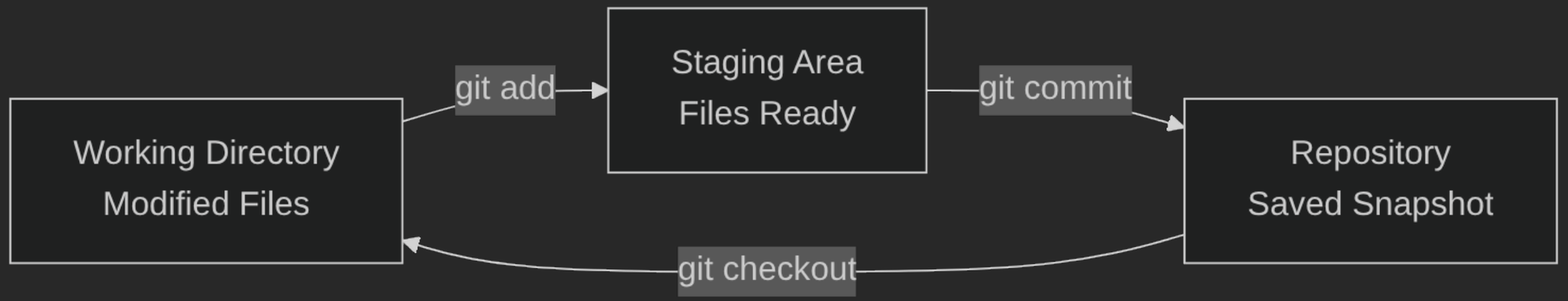
- Files you're editing
- Changes not yet staged

**2. Staging Area** (Index)

- Files marked to be committed
- Preview of next commit

**3. Repository** (History)

- Committed snapshots
- Permanent record




Think of it like:

- **Working Directory** = your desk (work in progress)
- **Staging Area** = package ready to ship
- **Repository** = warehouse (permanent storage)

---

# Git Command — git add

Stage files to be committed.

**Add specific file:**

```
git add app.py
```

**Add all files:**

```
git add .
```

**Add multiple files:**

```
git add app.py config.py utils.py
```

What happens:

- Files move from **Working Directory → Staging Area**
- Changes are **prepared** for commit
- You can review what will be committed

Example workflow:

```
# Edit app.py
vim app.py

# Stage the changes
git add app.py

# Check status
git status
```

---

# Git Command — git commit

Save a snapshot of staged changes.

**Basic commit:**

```
git commit -m "Add user authentication"
```

**Commit with description:**

```
git commit -m "Fix login bug

- Resolved issue with password validation
- Added error handling for empty fields"
```

What happens:

- Staged files are **saved** to repository
- A **commit** (snapshot) is created
- You get a **unique commit hash** (e.g., a1b2c3d)

**Best Practices:**

- Write **clear, descriptive** messages
- Commit **logical units** of work
- Commit **often** (small, frequent commits)

---

# Git Command — git status

Check the state of your repository.

```
git status
```

Shows:

- **Untracked files** — new files not added yet
- **Modified files** — changed but not staged
- **Staged files** — ready to commit
- **Branch name** — current branch

Example output:

```
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  modified:   app.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  config.py

Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)
  new file:   utils.py
```

---

# Git Command — git log

View commit history.

**Basic log:**

```
git log
```

**One-line format:**

```
git log --oneline
```

**Graphical view:**

```
git log --graph --oneline --all
```

Example output:

```
a1b2c3d (HEAD -> main) Add user authentication
e4f5g6h Fix login bug
h7i8j9k Initial commit
```

Each commit shows:

- **Hash** — unique identifier
- **Author** — who made the change
- **Date** — when it was made
- **Message** — what changed

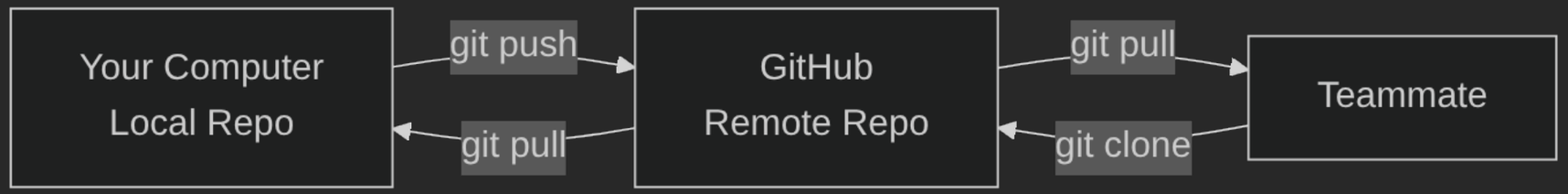
---

# What is GitHub?

**GitHub** is a **cloud-based hosting service** for Git repositories.

Think of it as:

- **Google Drive** for code
- **Social network** for developers
- **Collaboration platform** for teams
- **Portfolio** for your projects




**Key Features:**

- **Free hosting** for public repositories
- **Web interface** to browse code
- **Pull requests** for code review
- **Issues** for bug tracking
- **Actions** for CI/CD automation

---

# Git Command — git remote

Connect your local repo to GitHub.

**Add remote repository:**

```
git remote add origin https://github.com/username/repo-name.git
```

**View remotes:**

```
git remote -v
```

**Remove remote:**

```
git remote remove origin
```

**Change remote URL:**

```
git remote set-url origin https://github.com/username/new-repo.git
```

The name origin is a **convention** for the main remote repository. You can have multiple remotes (e.g., origin, upstream).

---

# Git Command — git push

Upload your commits to GitHub.

**Push to main branch:**

```
git push origin main
```

**First push (set upstream):**

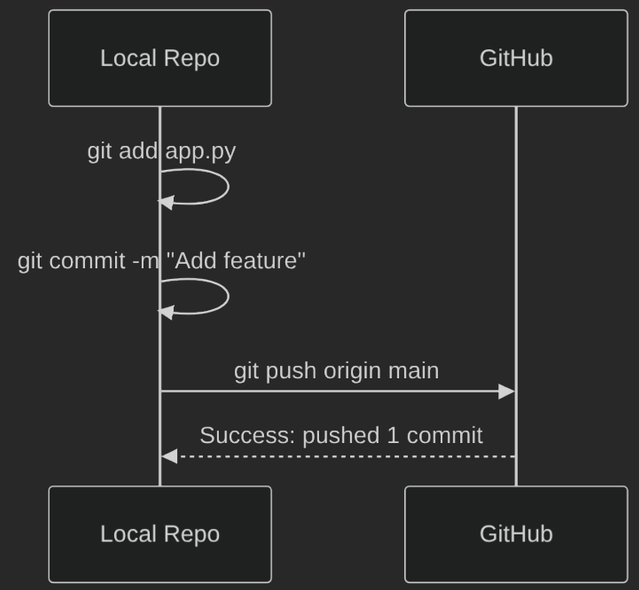
```
git push -u origin main
```

**Push all branches:**

```
git push --all origin
```

What happens:

- Local commits are **uploaded** to GitHub
- Others can now **see and pull** your changes
- Your code is **backed up** in the cloud



# Git Command — git pull

Download and merge changes from GitHub.

**Pull from main branch:**

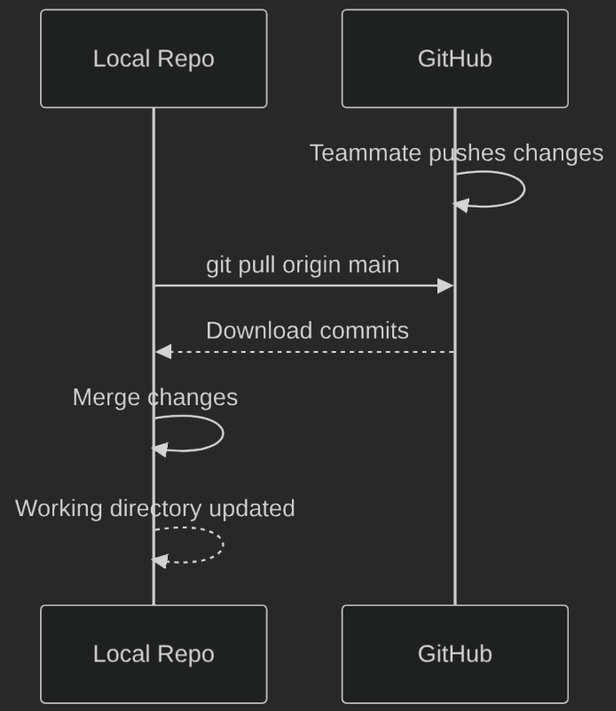
```
git pull origin main
```

**Pull and rebase:**

```
git pull --rebase origin main
```

What happens:

- **Fetches** latest commits from GitHub
- **Merges** them into your local branch
- Updates your working directory




**Always pull before pushing** to avoid conflicts!

# Git Command — git clone

Copy a repository from GitHub to your computer.

**Clone a repository:**

```
git clone https://github.com/username/repo-name.git
```

**Clone to specific folder:**

```
git clone https://github.com/username/repo-name.git my-project
```

**Clone with SSH:**

```
git clone git@github.com:username/repo-name.git
```

What happens:

- **Downloads** entire repository
- **Creates** local copy with full history
- **Sets up** remote connection automatically

This is how you:

- **Start** working on existing projects
- **Fork** open-source projects
- **Backup** repositories locally

---

# Complete Git Workflow

Typical daily workflow:

**Step-by-step:**

```
# 1. Get latest changes
git pull origin main

# 2. Make changes
vim app.py

# 3. Check what changed
git status

# 4. Stage changes
git add app.py

# 5. Commit
git commit -m "Add new feature"

# 6. Push to GitHub
git push origin main
```

---

# Practical Example — Setting Up a Project

Let's create a simple Python app and push it to GitHub.

**Step 1: Create project locally**

```
mkdir my-python-app
cd my-python-app
```

**Step 2: Initialize Git**

```
git init
```

**Step 3: Create app.py**

```python
def greet(name):
    """Greet a user by name."""
    return f"Hello, {name}! Welcome to Python."

def main():
    name = input("Enter your name: ")
    message = greet(name)
    print(message)

if __name__ == "__main__":
    main()
```

**Step 4: Create README.md**

```markdown
# My Python App

A simple greeting application.

## Usage

```bash
python app.py
```
```

---

# Practical Example – First Commit

**Step 5: Stage and commit**

```
# Add all files
git add .

# Check status
git status

# Make first commit
git commit -m "Initial commit: Add greeting app"
```

**Step 6: View history**

```
git log --oneline
```

Output:

```
a1b2c3d (HEAD -> main) Initial commit: Add greeting app
```

Your code is now **tracked locally** with Git!

---

# Practical Example — Push to GitHub

**Step 7: Create repository on GitHub**

1. Go to github.com
2. Click **"New repository"**
3. Name it my-python-app
4. **Don't** initialize with README (we already have one)
5. Click **"Create repository"**

**Step 8: Connect and push**

```
# Add GitHub as remote
git remote add origin https://github.com/yourusername/my-python-app.git

# Push to GitHub
git push -u origin main
```

**Step 9: Verify**

- Visit https://github.com/yourusername/my-python-app
- You should see app.py and README.md
- Your code is now **on GitHub**!

---

# Practical Example – Making Changes

**Step 10: Update the app**

Edit `app.py`:

```python
def greet(name, language="en"):
    """Greet a user by name in different languages."""
    greetings = {
        "en": f"Hello, {name}! Welcome to Python.",
        "es": f"¡Hola, {name}! Bienvenido a Python.",
        "fr": f"Bonjour, {name}! Bienvenue à Python."
    }
    return greetings.get(language, greetings["en"])

def main():
    name = input("Enter your name: ")
    lang = input("Language (en/es/fr): ").lower() or "en"
    message = greet(name, lang)
    print(message)

if __name__ == "__main__":
    main()
```

**Step 11: Commit and push changes**

```bash
git add app.py
git commit -m "Add multi-language support"
git push origin main
```

Your changes are now **on GitHub**!

# Git Configuration – First Time Setup

Before your first commit, configure Git:

**Set your name:**

```
git config --global user.name "Your Name"
```

**Set your email:**

```
git config --global user.email "your.email@example.com"
```

These settings are used in **every commit** to identify the author.

**Note:** Use the **same email** as your GitHub account for proper attribution.

**Check configuration:**

```
git config --list
```

**View specific setting:**

```
git config user.name
```

---

# Mini Task — Git Practice

Create an app (or reuse one) and manage it with Git:

**Part 1: Local Setup**

1. Create folder some-app
2. Initialize Git repository
3. Create some.py with basic operations
4. Create README.md with usage instructions
5. Make initial commit

**Part 2: GitHub Setup**

1. Create repository on GitHub
2. Connect local repo to GitHub
3. Push your code

**Part 3: Enhancements**

1. Add a new feature (e.g., power operation)
2. Commit the change
3. Push to GitHub
4. View your commit history on GitHub

**Part 4: Documentation**

1. Update README.md with the new feature
2. Commit and push
3. Verify changes on GitHub

**Goal:** Practice the complete Git workflow from local to GitHub.

---

# Thanks!

By ElnurBDa

# Databases & SQL

Python Course · Module 7

ElnurBDa

# Module 7 — Databases & SQL

In this module we connect **real-world programs** to **persistent, structured storage**.

We will learn:

- What databases are and when to use them
- Main database types
- How to use **SQLite** with Python
- Essential **SQL** syntax
- Building a simple database application

---

# Files vs. Databases

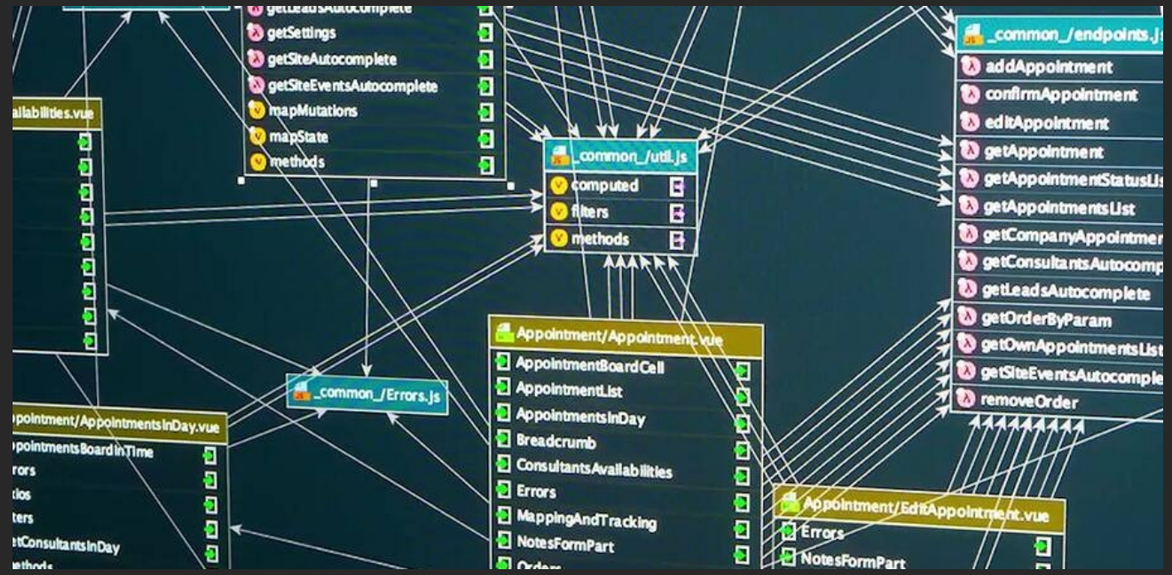
Programs normally store data in **RAM** (temporary).
Files and databases store data on **disk** (persistent).

**Files are good for:**

- Save logs
- Export reports
- Config files
- Simple data exchange

**Databases are good for:**

- Large data sets
- Structured relations
- Many users at once
- Fast searching / filtering



# What is a Database?

A **database** is an organized collection of data.

Key features:

- Structured storage (tables / documents / graphs)
- Fast querying (indexes)
- Concurrency (many users at once)
- Security (permissions, access control)

Compared to raw files:

- Databases **understand** what the data means
- Easier to enforce rules (constraints)
- Optimized for large datasets

---

# Main Database Types

**SQL / Relational**

- Tables: rows & columns
- Strong schema
- Uses SQL language
- Examples: PostgreSQL, MySQL, SQLite

Best for:

- Transactions
- Clear structure
- Reporting

**NoSQL**

- Flexible schema
- Designed for scale

Flavors:

- Key-value (Redis)
- Document (MongoDB)
- Graph (Neo4j)
- Wide-column (Cassandra)

---

# SQLite — Lightweight SQL

SQLite is a **serverless** database engine.

Characteristics:

- Everything in **one file** (e.g. `app.db`)
- Excellent for desktop / small tools / testing
- Comes built-in with Python (`sqlite3` module)
- No separate install, no server process

# Understanding Relational Databases — Tables, Rows & Columns

Think of a **table** like a **spreadsheet**:

| id | username | email | score |
|---|---|---|---|
| 1 | alice | alice@example.com | 100 |
| 2 | bob | bob@example.com | 85 |
| 3 | charlie | charlie@example.com | 92 |

- **Table** — the whole collection (like a sheet named "users")
- **Columns** — vertical categories (id, username, email, score)
- **Rows** — horizontal records (one per user)
- **Cell** — intersection of row & column (a single value)

---

# Relational Databases — Why Relations?

**Relational** means tables are connected.

Example — two related tables:

**USERS table:**

```
id | username | score
---+----------+-------
1  | alice    | 100
2  | bob      | 85
3  | charlie  | 92
```

**POSTS table:**

```
id | user_id | title
---+---------+------------
1  | 1       | "Hello"
2  | 1       | "Hi again"
3  | 2       | "World"
```

The `user_id` in POSTS points to `id` in USERS.

**Benefits:**

- No duplicate data (alice's info stored once)
- Easy to update (change once, everywhere updates)
- Enforced connections (`FOREIGN KEY`)
- Prevent invalid references (no post from user_id 999)

---

# Database Schema — Planning Your Data

Before creating tables, **plan your structure**:

1. **Identify entities** — What objects exist? (users, posts, comments)
2. **List attributes** — What info about each? (username, email, date)
3. **Choose types** — TEXT, INTEGER, REAL, etc.
4. **Define constraints** — Required? Unique? Default?
5. **Add relationships** — How do entities connect?

Example planning:

```
USERS:
- id (INTEGER, PRIMARY KEY, auto-increment)
- username (TEXT, NOT NULL, UNIQUE)
- email (TEXT, NOT NULL, UNIQUE)
- created_at (TEXT)

POSTS:
- id (INTEGER, PRIMARY KEY, auto-increment)
- user_id (INTEGER, FOREIGN KEY → users.id)
- title (TEXT, NOT NULL)
- content (TEXT)
- created_at (TEXT)
```

---

# Connecting to SQLite

```python
import sqlite3

# creates file if it does not exist
conn = sqlite3.connect("app.db")

print("Connected:", conn)

conn.close()
```

[finished]

```
Connected: <sqlite3.Connection object at
0x7f3a98956c50>
```

General pattern:

```python
conn = sqlite3.connect("app.db")
cur = conn.cursor()
# ... execute SQL here ...
conn.commit()
conn.close()
```

# SQL Basics — Data Types

SQLite supports common data types:

```
INTEGER    whole numbers (1, -42, 1000)
REAL       floating point (3.14, 2.71)
TEXT       strings ("hello", "John")
BLOB       binary data (images, files)
NULL       no value (missing data)
```

Example:

```sql
CREATE TABLE products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    price REAL,
    quantity INTEGER DEFAULT 0,
    description TEXT
);
```

---

# SQL — CREATE TABLE

```sql
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT NOT NULL UNIQUE,
    email TEXT NOT NULL UNIQUE,
    score INTEGER DEFAULT 0,
    created_at TEXT
);
```

Key constraints:

- PRIMARY KEY — unique identifier
- NOT NULL — value must be provided
- UNIQUE — no duplicate values
- DEFAULT — default value if not provided
- AUTOINCREMENT — auto-generate incrementing IDs

---

# SQL — INSERT

Insert new rows into a table.

```sql
INSERT INTO users (username, email, score)
VALUES ('alice', 'alice@example.com', 100);
```

Insert multiple rows:

```sql
INSERT INTO users (username, email, score)
VALUES
    ('bob', 'bob@example.com', 85),
    ('charlie', 'charlie@example.com', 92),
    ('diana', 'diana@example.com', 78);
```

Always specify column names for clarity and safety.

---

# SQL — SELECT

Query data from a table.

```sql
SELECT * FROM users;
```

Select specific columns:

```sql
SELECT username, score FROM users;
```

With WHERE condition:

```sql
SELECT * FROM users WHERE score > 80;
```

With multiple conditions:

```sql
SELECT * FROM users
WHERE score > 80 AND username LIKE 'a%';
```

---

# SQL — SELECT Advanced

**ORDER BY** — sort results:

```
SELECT username, score FROM users ORDER BY score DESC;
```

**LIMIT** — restrict number of results:

```
SELECT * FROM users LIMIT 5;
```

**OFFSET** — pagination:

```
SELECT * FROM users LIMIT 5 OFFSET 10;
```

**COUNT** — count rows:

```
SELECT COUNT(*) FROM users;
```

**Aggregate functions**: SUM(), AVG(), MIN(), MAX()

---

# SQL — UPDATE

Modify existing rows.

```
UPDATE users SET score = 150 WHERE username = 'alice';
```

Update multiple columns:

```
UPDATE users
SET score = 95, email = 'newemail@example.com'
WHERE username = 'bob';
```

Update with calculations:

```
UPDATE users SET score = score + 10 WHERE score < 50;
```

⚠️ **Always use WHERE clause** to avoid updating all rows!

---

# SQL — DELETE

Remove rows from a table.

```sql
DELETE FROM users WHERE username = 'alice';
```

Delete multiple rows:

```sql
DELETE FROM users WHERE score < 10;
```

Delete all rows (keep table structure):

```sql
DELETE FROM users;
```

⚠️ **Always use WHERE clause** — without it, all rows are deleted!

---

# SQL — Relationships & Foreign Keys

Create related tables:

```sql
CREATE TABLE posts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    title TEXT NOT NULL,
    content TEXT,
    FOREIGN KEY (user_id) REFERENCES users(id)
);
```

**FOREIGN KEY** ensures:

- Only valid user IDs can be referenced
- Data integrity
- Prevents orphaned posts

---

# SQL — JOIN

Combine data from multiple tables.

```
SELECT users.username, posts.title
FROM posts
JOIN users ON posts.user_id = users.id;
```

Different JOIN types:

- INNER JOIN — matching rows only
- LEFT JOIN — all from left table
- RIGHT JOIN — all from right table
- FULL OUTER JOIN — all from both tables

Example with filtering:

```
SELECT users.username, COUNT(posts.id) as post_count
FROM users
LEFT JOIN posts ON users.id = posts.user_id
GROUP BY users.id
HAVING COUNT(posts.id) > 0;
```

---

# SQLite CLI

SQLite has a **command-line tool** for quick testing.

Open a database:

```
sqlite3 app.db
```

You'll see:

```
SQLite version 3.x.x
sqlite>
```

Common commands:

```
.tables              -- list tables
.schema              -- show CREATE statements
.exit                -- quit
.mode column         -- prettier output
.headers on          -- show column names
```

---

# SQLite CLI — Running SQL

```
sqlite> CREATE TABLE users (
...>     id INTEGER PRIMARY KEY AUTOINCREMENT,
...>     username TEXT NOT NULL UNIQUE,
...>     score INTEGER DEFAULT 0
...> );

sqlite> INSERT INTO users (username, score) VALUES ('alice', 100);
sqlite> INSERT INTO users (username, score) VALUES ('bob', 85);

sqlite> SELECT * FROM users;
id|username|score
1|alice|100
2|bob|85

sqlite> SELECT * FROM users WHERE score > 80 ORDER BY score DESC;
id|username|score
1|alice|100
2|bob|85

sqlite> UPDATE users SET score = 95 WHERE username = 'bob';

sqlite> .exit
```

Tip: Press ↑ to recall previous commands!

# Using SQL in Python

```python
import sqlite3

conn = sqlite3.connect("app.db")
cur = conn.cursor()

sql = (
    "CREATE TABLE IF NOT EXISTS users ("
    " id INTEGER PRIMARY KEY AUTOINCREMENT,"
    " username TEXT NOT NULL UNIQUE,"
    " score INTEGER DEFAULT 0"
    ")"
)
cur.execute(sql)

conn.commit()
conn.close()
print("Table created.")
```

[finished]

```
Table created.
```

Notes:

- IF NOT EXISTS avoids errors
- PRIMARY KEY gives unique id

# CRUD — Insert & Select

```python
import sqlite3

conn = sqlite3.connect("app.db")
cur = conn.cursor()

cur.execute(
    "INSERT INTO users (username, score) VALUES (?, ?)",
    ("admin", 100),
)

cur.execute("SELECT id, username, score FROM users")
rows = cur.fetchall()

for row in rows:
    print(row)

conn.commit()
conn.close()
```

——————————— [finished with error] ———————————

```
Traceback (most recent call last):
  File "/tmp/nix-shell-21250-0/.presentermmZNakj/snippet.py", line 6, in
<module>
    cur.execute(
    ~~~~~~~~~~~^
        "INSERT INTO users (username, score) VALUES (?, ?)",
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
        ("admin", 100),
        ^^^^^^^^^^^^^^^
    )
    ^
sqlite3.IntegrityError: UNIQUE constraint failed: users.username
```

? placeholders prevent SQL injection.

# CRUD — Update & Delete

```python
import sqlite3

conn = sqlite3.connect("app.db")
cur = conn.cursor()

cur.execute(
    "UPDATE users SET score = ? WHERE username = ?",
    (150, "admin"),
)

cur.execute(
    "DELETE FROM users WHERE username = ?",
    ("guest",),
)

conn.commit()
conn.close()
```

——————————— [finished] ———————————

Always `commit()` after changes.

# Putting It Together

```python
import sqlite3

def get_conn():
    return sqlite3.connect("app.db")

def add_user(username, score):
    with get_conn() as conn:
        cur = conn.cursor()
        cur.execute(
            "INSERT INTO users (username, score) VALUES (?, ?)",
            (username, score),
        )

def list_users():
    with get_conn() as conn:
        cur = conn.cursor()
        cur.execute("SELECT id, username, score FROM users")
        return cur.fetchall()
```

Pattern:

- Small helper functions
- Use `with` for connection lifetime

---

# Mini Task

Create a mini **score tracker**:

1. Create SQLite DB & users table
2. Ask user for username and score
3. Insert using **parameterized** query
4. Select and print all users
5. Add option to **update** score

Bonus:

- Create a posts table
- Link posts to users with FOREIGN KEY
- Show user + their posts

# Thanks!

By ElnurBDa

# Databases & SQL Practice

Python Course · Module 7 Practice

ElnurBDa

# Practice 1 — Student Score Tracker

Create score_tracker.py that uses SQLite to manage student scores with full CRUD operations.

**Requirements:**

1. Create a database file scores.db with a table students:

```
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    score INTEGER NOT NULL,
    subject TEXT NOT NULL,
    created_at TEXT
);
```

2. Implement a menu system with a while loop:
   - 1. Add student score
   - 2. View all scores
   - 3. Update score
   - 4. Delete score
   - 5. View scores by subject
   - 6. Calculate average score
   - 7. Exit
3. For **Add student score**:
   - Ask for name, score (0-100), and subject
   - Validate score is between 0 and 100
   - Use **parameterized queries** with ? placeholders
   - Add current timestamp to created_at
4. For **View all scores**:
   - Display all students with their scores and subjects
   - Format output nicely (e.g., "1. Alice - Math: 95")
5. For **Update score**:
   - Ask for student ID and new score
   - Validate score range
   - Use parameterized UPDATE query
6. For **Delete score**:
   - Ask for student ID
   - Confirm deletion before proceeding
   - Use parameterized DELETE query
7. For **View scores by subject**:
   - Ask for subject name
   - Filter and display only scores for that subject
   - Use WHERE clause with parameterized query
8. For **Calculate average score**:
   - Calculate and display the average of all scores
   - Use SQL AVG() function or calculate in Python

**Example interaction:**

```
Menu:
1. Add student score
2. View all scores
3. Update score
4. Delete score
5. View scores by subject
6. Calculate average score
7. Exit

Choice: 1
Enter student name: Alice
Enter score (0-100): 95
Enter subject: Math
Score added successfully!

Choice: 2
All Scores:
1. Alice - Math: 95
2. Bob - Science: 87
3. Alice - Science: 92

Choice: 5
Enter subject: Math
Scores for Math:
1. Alice - Math: 95

Choice: 6
Average score: 91.33
```

**Important:** Always use **parameterized queries** (? placeholders) to prevent SQL injection.

---

# Practice 2 — Library Database with Relationships

Create library_db.py that implements a relational database with **two related tables** using foreign keys.

**Requirements:**

1. Create library.db with two tables:
1. **Books table:**

```
CREATE TABLE IF NOT EXISTS books (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    author TEXT NOT NULL,
    isbn TEXT UNIQUE,
    available INTEGER NOT NULL DEFAULT 1
);
```

**Borrowers table:**

```
CREATE TABLE IF NOT EXISTS borrowers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    book_id INTEGER NOT NULL,
    borrower_name TEXT NOT NULL,
    borrowed_date TEXT NOT NULL,
    returned_date TEXT,
    FOREIGN KEY (book_id) REFERENCES books(id)
);
```

2. Implement helper functions:
   - init_database() - creates both tables if they don't exist
   - add_book(title, author, isbn) - inserts a new book
   - list_books() - shows all books with availability status
   - borrow_book(book_id, borrower_name) - creates a borrower record and marks book as unavailable
   - return_book(book_id) - updates borrower record with return date and marks book as available
   - view_borrowing_history(book_id) - shows all borrowers for a specific book
3. Implement a menu system:
   - 1. Add book
   - 2. List all books
   - 3. Borrow book
   - 4. Return book
   - 5. View borrowing history
   - 6. View all active borrows
   - 7. Exit
4. For **Borrow book**:
   - Show available books first
   - Ask for book ID and borrower name
   - Check if book is available (use WHERE clause)
   - If available, insert into borrowers and update books.available = 0
   - If not available, show error message
5. For **Return book**:
   - Show currently borrowed books
   - Ask for book ID
   - Update borrowers.returned_date and set books.available = 1
6. For **View borrowing history**:
   - Ask for book ID
   - Use a **JOIN** query to show book title and all borrower records:

```
SELECT books.title, borrowers.borrower_name,
        borrowers.borrowed_date, borrowers.
returned_date
FROM borrowers
JOIN books ON borrowers.book_id = books.id
WHERE books.id = ?
```

7. For **View all active borrows**:
   - Use JOIN to show books that are currently borrowed (where returned_date IS NULL)

**Important:**

- Always use **parameterized queries**
- Handle cases where book doesn't exist or is already borrowed
- Use **JOIN** queries to combine data from both tables

**Example interaction:**

```
Menu:
1. Add book
2. List all books
3. Borrow book
4. Return book
5. View borrowing history
6. View all active borrows
7. Exit

Choice: 1
Enter book title: Python Basics
Enter author: John Doe
Enter ISBN: 123456789
Book added!

Choice: 2
All Books:
1. Python Basics by John Doe (ISBN: 123456789) -
Available
2. Advanced Python by Jane Smith (ISBN:
987654321) - Available

Choice: 3
Available Books:
1. Python Basics - Available
2. Advanced Python - Available
Enter book ID to borrow: 1
Enter borrower name: Alice
Book borrowed successfully!

Choice: 6
Active Borrows:
- Python Basics borrowed by Alice on 2025-01-15

Choice: 5
Enter book ID: 1
Borrowing History for "Python Basics":
- Alice borrowed on 2025-01-15, returned on
2025-01-20
- Bob borrowed on 2025-01-25, not yet returned
```

# Web Basics

Python Course · Module 8

ElnurBDa

# Module 8 — Web Basics

We connect Python skills to the web stack:

We will go in three stages:

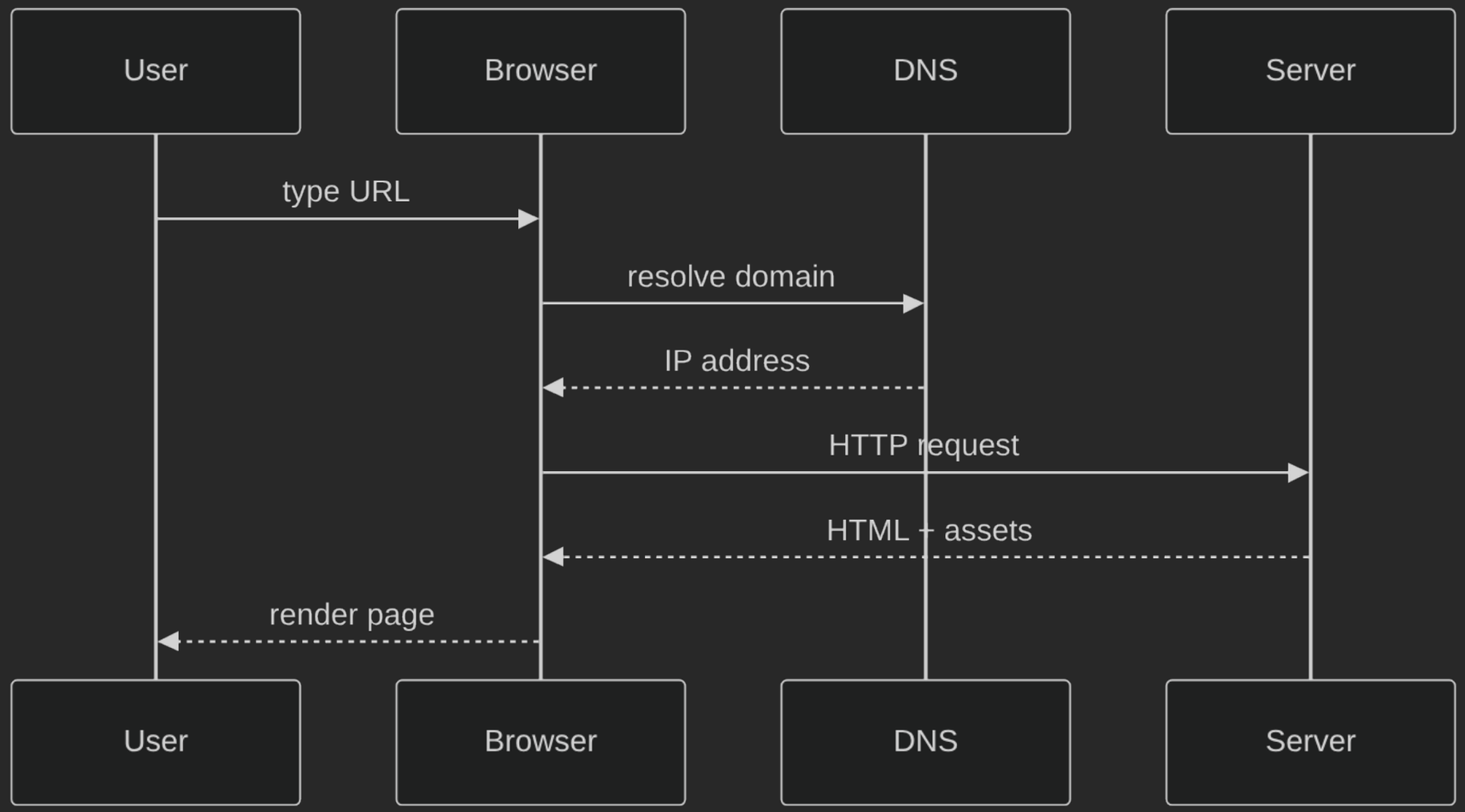
1. **Web pages** — HTML, CSS, JavaScript, and how the browser talks to a server
2. **HTTP & APIs** — requests, responses, status codes, and JSON
3. **Python as a web client/server** — using requests and serving simple pages

---

# How the Web Works

Step-by-step workflow from browser to server:

1. User enters https://example.com
2. Browser resolves domain → IP
3. Browser sends HTTP request
4. Server responds with HTML, CSS, JS
5. Browser renders page



# Section 1 — Web Pages (HTML, CSS, JS)

In this first part we focus on **how a page is built and served**. Do not worry about APIs or JSON yet — just think in terms of:

- HTML = structure & content
- CSS = visual style
- JavaScript = interactivity in the browser

Later sections will reuse this when we load data from APIs instead of hard-coding it.

---

# HTML — Building Blocks

HTML describes structure/content of web pages (like blueprints of a house).

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Hello Web</title>
  </head>
  <body>
    <h1>Welcome!</h1>
    <p>This is my first web page.</p>
    <a href="https://python.org">Python.org</a>
  </body>
</html>
```

- Tags usually come in pairs: `<tag>content</tag>`
- Attributes add metadata: `<a href="https://python.org">`
- Browser builds the **DOM tree** from these nested tags

---

# HTML Storyboard: Code → Server → Browser

1. **Write HTML** (index.html)
2. **Serve it** (python3 -m http.server 8000)
3. **View in browser** (http://localhost:8000)

```
[index.html] --(http.server)--> [localhost:8000] --(render)--> [Page]
```

Focus questions:

- Do tags nest correctly? (<main> contains <section>)
- Did you include alt text for images?
- Can you describe each section aloud?

---

# HTML Document Anatomy

| Section | Purpose |
|---|---|
| <!DOCTYPE> | Tells browser to use modern HTML5 rules |
| <html> | Root element wrapping everything |
| <head> | Metadata, title, linked CSS/JS |
| <body> | Visible content: text, images, forms |

Example with more tags:

```html
<body>
  <header><h1>Cyber Café</h1></header>
  <main>
    <section>
      <h2>Menu</h2>
      <ul>
        <li><strong>Espresso</strong> — 3 AZN</li>
        <li><strong>Tea</strong> — 2 AZN</li>
      </ul>
    </section>
    <img src="assets/cafe.jpg" alt="Cafe interior" />
  </main>
  <footer>&copy; 2025 Cyber Café</footer>
</body>
```

# Core HTML Tags & Patterns

| Tag | What it does | Example snippet |
|---|---|---|
| `<h1>` | Page title | `<h1>News</h1>` |
| `<p>` | Paragraph | `<p>Breaking updates...</p>` |
| `<a>` | Link to another page/file | `<a href="/about.html">About</a>` |
| `<img>` | Display image | `<img src="logo.png" alt="Logo" />` |
| `<ul>/<ol>` | Lists (unordered/ordered) | `<ul><li>Item</li></ul>` |
| `<form>` | User input | `<form><input name="email" /></form>` |
| `<div>/<span>` | Layout/grouping | `<div class="card">...</div>` |

Tip: Start simple, add CSS later to style it.

# Styling with CSS

CSS (Cascading Style Sheets) controls presentation.

Ways to include CSS:

- Inline: `<p style="color:red">`
- `<style>` block in `<head>`
- External file: `<link rel="stylesheet" href="styles.css" />`

Example:

```html
<head>
  <link rel="stylesheet" href="assets/styles.css" />
</head>
<body>
  <h1 class="hero">Welcome</h1>
</body>
```

```css
.hero {
  color: #ff9900;
  text-align: center;
  border-bottom: 2px solid #444;
}
```

Guideline: keep structure (HTML) separate from style (CSS) for clarity.

---

# JavaScript — Introduction

JavaScript is a **programming language** that runs in web browsers (and servers).

Key characteristics:

- **Dynamic** — variables can change types
- **Interpreted** — runs directly without compilation
- **Event-driven** — responds to user actions (clicks, typing, etc.)
- **Versatile** — used for both frontend (browser) and backend (Node.js)

Why JavaScript matters:

- Makes web pages **interactive**
- Can update page content without reloading
- Communicates with servers (APIs)
- Essential for modern web development

Think of it as the "brain" that makes static HTML pages come alive.

# JavaScript Basics — Variables & Types

JavaScript has similar concepts to Python, with different syntax.

**Variables:**

```javascript
let name = "Alice";        // can be reassigned
const age = 25;            // constant, cannot be reassigned
var oldStyle = "avoid";     // old way, avoid in modern code
```

**Data Types:**

```javascript
let text = "Hello";         // string
let number = 42;            // number
let isActive = true;        // boolean
let data = null;            // null (no value)
let items = [1, 2, 3];      // array (like Python list)
let user = {                // object (like Python dict)
    name: "Alice",
    age: 25
};
```

**Functions:**

```javascript
function greet(name) {
    return "Hello, " + name;
}

// Arrow function (modern style)
const greet2 = (name) => {
    return "Hello, " + name;
};
```

---

# JavaScript vs Node.js

**JavaScript (Browser)**

- Runs in the **browser**
- Can manipulate HTML/CSS
- Access to document, window objects
- Cannot access file system (security)
- Used for: interactive web pages, form validation, animations

Example:

```
// Runs in browser
document.getElementById("btn").onclick = function() {
    alert("Clicked!");
};
```

**Node.js (Server)**

- JavaScript runtime for **servers**
- Built on Chrome's V8 engine
- Can access file system, databases
- No document or window objects
- Used for: web servers, APIs, command-line tools

Example:

```
// Runs on server (Node.js)
const fs = require('fs');
fs.readFile('data.txt', 'utf8', (err, data) => {
    console.log(data);
});
```

**Key Point:** Same language, different environments. In this module, we focus on **browser JavaScript**.

---

# JavaScript in HTML

JavaScript brings interactivity to web pages.

**Ways to include JavaScript:**

1. **Inline** (simple actions):

```
<button onclick="alert('Hello!')">Click me</button>
```

2. **Script tag** (inside HTML):

```
<script>
    console.log("Page loaded!");
</script>
```

3. **External file** (recommended):

```
<script src="app.js"></script>
```

**Simple example:**

```
<body>
  <p id="message">Click the button</p>
  <button onclick="changeText()">Click me</button>

  <script>
    function changeText() {
      document.getElementById("message").textContent = "Hello from JavaScript!";
    }
  </script>
</body>
```

Place <script> tags near the end of <body> so HTML loads first.

---

# JavaScript — Practical Example

Here's a more complete example that demonstrates common patterns:

```html
<body>
  <h1>Counter App</h1>
  <p>Count: <span id="count">0</span></p>
  <button id="increment">+</button>
  <button id="decrement">-</button>
  <button id="reset">Reset</button>

  <script>
    let count = 0;
    const countDisplay = document.getElementById("count");
    const incrementBtn = document.getElementById("increment");
    const decrementBtn = document.getElementById("decrement");
    const resetBtn = document.getElementById("reset");

    function updateDisplay() {
      countDisplay.textContent = count;
    }

    incrementBtn.addEventListener("click", () => {
      count++;
      updateDisplay();
    });

    decrementBtn.addEventListener("click", () => {
      count--;
      updateDisplay();
    });

    resetBtn.addEventListener("click", () => {
      count = 0;
      updateDisplay();
    });
  </script>
</body>
```

**What's happening:**

1. **Variables** (`count`) store state
2. `getElementById()` finds HTML elements
3. `addEventListener()` responds to clicks
4. `textContent` updates what's displayed
5. **Functions** organize the logic

This pattern (select element → listen for events → update content) is the foundation of interactive web pages.

# Serving index.html Quickly

Browsers look for index.html by default.

```
cd website_project
python3 -m http.server 8000
```

- Place index.html alongside the command
- Visit http://localhost:8000 to see it rendered
- Great for checking static prototypes before moving to frameworks

Challenge: create index.html with header, paragraph, list, and image, then serve it locally.

---

# Section 2 — HTTP & APIs

Now that you know what a **page** looks like, we zoom in on the **conversation** between browser and server.

Questions to keep in mind:

- Who is the **client** in this example? Who is the **server**?
- What is the **URL**, what is the **method** (GET/POST/...), and what is inside the **body** (if any)?
- Is the response returning **HTML** or **JSON**?

Once this feels comfortable, we will replace “browser” with **Python code** using requests.

---

# HTTP — The Conversation Rules

HTTP (HyperText Transfer Protocol) defines how clients & servers talk — like a script both sides follow.

Structure:

```
<METHOD> <PATH> HTTP/1.1
Header: Value
Header: Value

<optional body>
```

Request example:

```
GET /api/users HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Accept: application/json
```

Response example:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"status": "ok", "count": 2}
```

Common verbs:

- GET (read), POST (create), PUT/PATCH (update), DELETE (remove)

---

# HTTP Anatomy — Request vs Response

**Request**

```
POST /login HTTP/1.1
Host: mysite.com
Content-Type: application/json
Authorization: Bearer <token>

{"email": "user@example.com", "password": "***"}
```

- Method + path
- Headers describe metadata (auth, format)
- Body carries data (optional)

**Response**

```
HTTP/1.1 200 OK
Content-Type: application/json
Set-Cookie: session=abc123

{"status": "ok", "message": "Welcome"}
```

- Status line communicates result
- Headers for caching, cookies, etc.
- Body returns JSON/HTML/binary

---

# Status Codes

| Code | Meaning | Use case |
|---|---|---|
| 200 | OK | Successful request |
| 201 | Created | Resource created (POST) |
| 301 | Moved Permanently | Redirect |
| 400 | Bad Request | Invalid client data |
| 401 | Unauthorized | Missing/invalid auth |
| 404 | Not Found | Resource missing |
| 500 | Internal Server Error | Server bug/problem |

Fun reference with cat photos: https://http.cat

# APIs — Digital Waiters

API = Application Programming Interface.

- Think of a restaurant: you (client) tell the waiter (API) what you want, they talk to the kitchen (server), and bring back the dish (data).
- HTTP APIs expose data/actions via URLs, verbs, and JSON payloads.
- Good APIs document their menu (endpoints, parameters, status codes).

Example: Tasks API served from http://localhost:3000

| Endpoint | Method | Description |
|---|---|---|
| /api/tasks?completed=true | GET | List tasks filtered by completion flag |
| /api/tasks | POST | Create a new task with title/description flags |
| /api/tasks/{id} | GET | Fetch one task by its numeric identifier |
| /api/tasks/{id} | PUT | Update title, description, or completed status |
| /api/tasks/{id} | DELETE | Remove the task forever |
| /api/health | GET | Quick health check for the service |

Menu analogy:

- Endpoint = dish name
- Query params/body = customization
- Response JSON = meal you receive

---

# JSON — Data Containers

JSON (JavaScript Object Notation) represents structured data with nested objects/lists.

```
{
  "id": 42,
  "username": "admin",
  "scores": [99, 88, 91],
  "profile": {
    "city": "Baku",
    "active": true
  }
}
```

Analogy: dictionaries + lists combined.

Rules of thumb:

- Keys in double quotes, values can be string/number/bool/null/object/array
- Order usually not important, whitespace ignored

Another sample (array of objects):

```
[
  {"task": "learn HTML", "done": true},
  {"task": "build API client", "done": false}
]
```

Analogy: shipping boxes inside a truck — arrays hold boxes, each box (object) has labeled compartments (keys).

---

# From HTTP & JSON to Python Code

Let’s connect the dots:

1. Browser or client sends an **HTTP request** (method + path + headers + body).
2. Server sends back an **HTTP response** with a **status code** and **body**.
3. If `Content-Type: application/json`, the body contains **JSON**, which feels like Python `dict` + `list`.
4. Our Python script with `requests` plays the role of the **client** instead of the browser.

So when you see `response.json()` later, imagine:

> “Take the JSON body from the HTTP response and turn it into Python dictionaries/lists I can work with.”

---

# Inspecting APIs

Tools:

- Browser dev tools → Network tab
- `curl` in terminal:

```
curl -G "http://localhost:3000/api/tasks" \
  --data-urlencode "completed=true"
```

- Postman, Bruno, or Insomnia for graphical testing

Always check: URL, method, headers, body, response code, payload. Tasks API also requires you to pass `completed` as a query parameter when listing.

---

# Python requests Basics

```python
import requests

url = "http://localhost:3000/api/tasks"
params = {"completed": "false"}  # API requires this flag
response = requests.get(url, params=params, timeout=5)

print("Status:", response.status_code)
response.raise_for_status()
tasks = response.json()
print("First task title:", tasks[0]["title"] if tasks else "No tasks yet")
```

snippet +exec is disabled, run with -x to enable

Notes:

- `timeout` prevents hanging forever (network problems)
- `response.status_code` shows the HTTP status (200, 404, 500, …)
- `response.raise_for_status()` throws an exception for 4xx/5xx errors
- `response.text` is the raw string body, `response.json()` parses JSON into Python types

---

# Checkpoint — Your First API Call

1. Activate your venv
2. pip install requests
3. Run:

```python
import requests

BASE_URL = "http://localhost:3000/api/tasks"
data = requests.get(
    BASE_URL,
    params={"completed": "true"},
    timeout=5,
).json()

print("Total completed tasks:", len(data))
print("Sample record:", data[0] if data else "None")
```

4. Screenshot output for homework

Questions to consider:

- What happens if you forget .json()?
- How would you handle an empty list or missing key?

---

# Section 3 — Python as Web Client & Simple Server

In this final part we:

1. Use Python + requests as an **HTTP client** talking to the Tasks API.
2. Use http.server (and a tiny custom server) as a **simple HTTP server** to serve static HTML.

Key mental model:

- Browser and Python scripts are just **different HTTP clients**.
- Servers (your simple Python server, or the Tasks API on port 3000) all speak the same HTTP “language”.

When in doubt, go back to the HTTP request/response examples and map each line to what requests or the browser is doing.

---

# Installing requests Inside a venv

```python3 -m venv .venv
source .venv/bin/activate  # Windows: .venv\Scripts\activate
pip install --upgrade pip
pip install requests
pip freeze
```

- Always isolate dependencies per project
- `pip freeze` helps document versions (`requirements.txt`)
- Deactivate with `deactivate` when done

---

# Sending Data with requests

```python
import requests

payload = {
    "title": "Draft Module 8 slides",
    "description": "Cover Tasks API examples",
    "completed": False,
}

resp = requests.post(
    "http://localhost:3000/api/tasks",
    json=payload,
    timeout=5,
)

resp.raise_for_status()
created = resp.json()
print("New task id:", created["id"])
print("Completed flag:", created["completed"])
```

snippet +exec is disabled, run with -x to enable

`json=...` auto converts dict to JSON & sets `Content-Type: application/json`.

# Serving Static HTML with Python

Quick demo using built-in http.server.

```
python3 -m http.server 8000
```

- Serves current directory
- Visit http://localhost:8000 to view files
- Great for testing HTML/CSS locally

Stop with Ctrl+C.

---

# Custom Minimal Server

Serve a specific HTML file with Python.

```python
from http.server import SimpleHTTPRequestHandler, HTTPServer
from pathlib import Path

PORT = 8080
ROOT = Path("website")

class StaticHandler(SimpleHTTPRequestHandler):
    def translate_path(self, path):
        # Map requests into our ROOT directory
        return str(ROOT / path.lstrip("/"))

if __name__ == "__main__":
    ROOT.mkdir(exist_ok=True)
    httpd = HTTPServer(("0.0.0.0", PORT), StaticHandler)
    print(f"Serving on http://localhost:{PORT}")
    httpd.serve_forever()
```

snippet +exec is disabled, run with -x to enable

Place `index.html` inside `website/` and run script.

Explanation:

- `HTTPServer` listens on port 8080 and waits for requests
- `SimpleHTTPRequestHandler` knows how to serve files; we override `translate_path` to force it into `website/`
- `ROOT.mkdir(exist_ok=True)` ensures the folder exists before serving
- Visit http://localhost:8080/index.html to test

Project structure reference:

```
website/
├── index.html
├── about.html
└── assets/
    └── styles.css
server.py
```

Run `python server.py` from the folder containing `website/`.

---

# Meme Slide

# Thanks!

By ElnurBDa

# Web Apps in Practice

Python Course · Module 9

ElnurBDa

# Module 9 — Web App Architecture & To-Do App

In this module we connect everything together in a **simple web application**:

- Basic web application architecture
- Frontend vs backend vs database
- Building a minimal backend for a To-Do app
- Building a minimal frontend for it
- Extra mini-project for practice

---

# Big Picture — Web Application Architecture

Modern web apps are usually split into three main parts:

- **Frontend** — code running in the browser (HTML/CSS/JS)
- **Backend** — server code handling requests and business logic
- **Database** — persistent storage for data

The browser never talks to the database directly — it always goes through the backend.

---

# Roles of Frontend, Backend, Database

| Layer | Main role | Technologies (examples) |
|---|---|---|
| Frontend | UI, layout, user interactions | HTML, CSS, JavaScript |
| Backend | Logic, validation, security, APIs | Python, Node, Java, Go, etc. |
| Database | Store and query structured data | SQLite, PostgreSQL, MySQL, etc. |

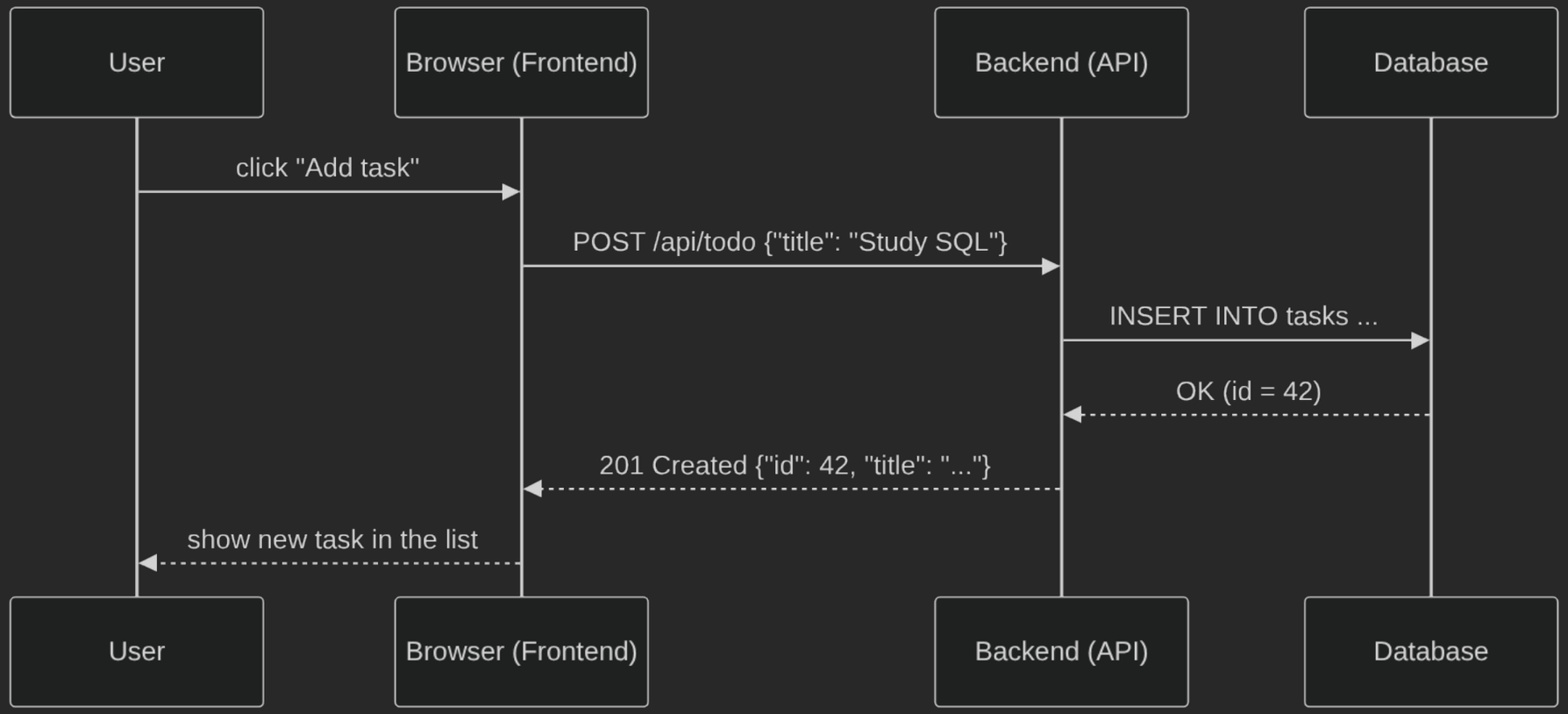
Analogy:

- Frontend = restaurant dining area & menu
- Backend = kitchen + waiters
- Database = pantry where ingredients are stored

---

# Request / Response Flow

User action → network request → backend → database → response → UI update.




Key idea: **HTTP** is the language between frontend and backend.

# Designing a Simple To-Do Backend

We will build a tiny **JSON API** in Python for a To-Do app.

Features:

- List all tasks
- Create new task
- Mark task as completed

Simplest storage option:

- Start with an **in-memory list** (lost on restart but easy to understand)
- Later you can swap it with SQLite (from Module 7)

---

# To-Do Backend — Minimal API (Flask)

todo_backend.py

```python
from flask import Flask, jsonify, request

app = Flask(__name__)

tasks = []  # in-memory list of dicts
next_id = 1

@app.get("/api/todo")
def list_tasks():
    return jsonify(tasks)

@app.post("/api/todo")
def create_task():
    global next_id
    data = request.get_json() or {}
    title = data.get("title", "").strip()
    if not title:
        return {"error": "title is required"}, 400

    task = {"id": next_id, "title": title, "completed": False}
    next_id += 1
    tasks.append(task)
    return task, 201

if __name__ == "__main__":
    app.run(debug=True)
```

snippet +exec is disabled, run with -x to enable

Run with:

```
pip install flask
python todo_backend.py
```

---

# Testing the Backend with curl

```
# list tasks
curl http://localhost:5000/api/todo

# create a new task
curl -X POST http://localhost:5000/api/todo \
  -H "Content-Type: application/json" \
  -d '{"title": "Finish Module 9 slides"}'
```

Check:

- HTTP method (GET, POST)
- URL path (/api/todo)
- Request body and headers
- Response JSON and status code

---

# Optional: Persist To-Dos with SQLite

You can reuse **Module 7** knowledge to save tasks in app.db.

Sketch:

```sql
CREATE TABLE IF NOT EXISTS tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    completed INTEGER NOT NULL DEFAULT 0
);
```

```python
import sqlite3

def get_conn():
    return sqlite3.connect("app.db")
```

Then inside your Flask handlers, instead of using a global tasks list, execute INSERT, SELECT, and UPDATE queries.

---

# Building the Frontend — HTML Skeleton

index.html

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Mini To-Do App</title>
    <style>
      body { font-family: system-ui, sans-serif; margin: 2rem; background: #1d2021; color: #ebdbb2; }
      h1 { color: #fabd2f; }
      input, button { padding: 0.4rem 0.6rem; margin-right: 0.4rem; }
      ul { list-style: none; padding: 0; }
      li { padding: 0.25rem 0; }
    </style>
  </head>
  <body>
    <h1>Mini To-Do App</h1>
    <input id="title" placeholder="New task..." />
    <button id="add-btn">Add</button>
    <ul id="list"></ul>

    <script src="app.js"></script>
  </body>
  </html>
```

Serve this folder with:

```bash
python3 -m http.server 8000
```

---

# Frontend Logic — Fetching Tasks

app.js

```javascript
const API_URL = "http://localhost:5000/api/todo";

async function loadTasks() {
  const res = await fetch(API_URL);
  const data = await res.json();

  const list = document.getElementById("list");
  list.innerHTML = "";
  for (const task of data) {
    const li = document.createElement("li");
    li.textContent = `${task.id}. ${task.title}`;
    list.appendChild(li);
  }
}

loadTasks();
```

Open http://localhost:8000 in a browser to see the list (empty at first).

---

# Frontend Logic — Creating Tasks from UI

Extend `app.js`:

```
const input = document.getElementById("title");
const button = document.getElementById("add-btn");

button.addEventListener("click", async () => {
  const title = input.value.trim();
  if (!title) return;

  await fetch(API_URL, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title }),
  });

  input.value = "";
  await loadTasks();
});
```

Flow:

- User types a title and clicks **Add**
- Browser sends `POST` request to backend
- Backend adds task and returns JSON
- Frontend reloads the list using `loadTasks()`

---

# Bringing It All Together

End-to-end:

1. Start backend:
   - `python todo_backend.py` → serves `http://localhost:5000/api/todo`
2. Start static frontend server in same folder as `index.html`:
   - `python3 -m http.server 8000`
3. Visit `http://localhost:8000` in browser:
   - Add tasks from UI
   - Confirm requests in **Network** tab of DevTools
4. Observe:
   - Frontend ↔ Backend via HTTP
   - Backend may use in-memory list or SQLite

You just built a tiny full-stack application.

---

# Mini Project — Student Manager

Build a mini student management tool that reuses the same architecture:

1. **Backend API**:
   - Endpoints: GET /api/students, POST /api/students, PATCH /api/students/<id>
   - Each student record includes id, name, email, course, and status (active/inactive)
2. **Database (recommended)**:
   - SQLite table students with columns (id INTEGER PRIMARY KEY, name TEXT, email TEXT UNIQUE, course TEXT, status TEXT)
   - Seed 3 demo rows for quick testing
3. **Frontend**:
   - Table showing all students (name, course, status badge)
   - Form to add a new student (name, email, course)
   - Button to toggle status between Active / Inactive (calls PATCH)

Stretch ideas:

- Filter students by course or status
- Add simple validation and inline error messages
- Export the current list to JSON for download

---

# Thanks!

By ElnurBDa

# Secure Python Basics

Python Course · Module 10

ElnurBDa

# Module 10 — Security & Privacy Basics

In this module we connect your Python, database, and web skills to **security**:

- Trust boundaries & input validation
- Common pitfalls in CLI, database, and web apps
- SQL injection and **parameterized queries**
- Handling secrets and sensitive data (passwords, tokens, logs)

Goal: build a **security mindset** for all the code you write next.

---

# Why Security Matters in Everyday Code

Even “small” scripts can:

- Delete or leak files
- Corrupt databases
- Expose personal data (names, emails, grades)

Realistic risks:

- Users type unexpected / malicious input
- Someone calls your API with weird parameters
- Logs or config files accidentally contain passwords or tokens

Security is not just about hacking tools — it’s about **writing code that behaves safely** under bad input.

---

# Trust Boundaries — Who Do You Trust?

Every program has **trust boundaries**:

- Data you fully control (constants, internal helpers)
- Data from **users** (CLI `input()`, HTML forms, API requests)
- Data from **external systems** (files, databases, 3rd-party APIs)

Rule of thumb:

> Anything that crosses a boundary into your program is **untrusted** until checked.

We'll see how this affects:

- Input validation
- SQL queries
- Logging and error messages

---

# Validating User Input (CLI)

Example: simple CLI for age input.

**Unsafe style** (no checks, will crash on bad data):

```
age = int(input("Enter your age: "))
print("Next year:", age + 1)
```

**Safer style** (validate + handle errors):

```
def ask_age():
    text = input("Enter your age: ").strip()
    if not text.isdigit():
        print("Please enter only digits.")
        return None

    age = int(text)
    if age <= 0 or age > 130:
        print("Please enter a realistic age.")
        return None

    return age

age = ask_age()
if age is not None:
    print("Next year you will be", age + 1)
```

snippet +exec is disabled, run with -x to enable

Pattern:

- **Validate** format and range
- **Fail gracefully** with a clear message

---

# Validating Web/API Input (Concept)

In web apps (like your Module 9 To-Do or Student Manager):

- Every field from a form or JSON body is **untrusted**.
- You must check: **type**, **length**, **allowed characters**, **business rules**.

Examples of validation rules:

- `username`: 3–32 chars, letters/numbers/underscores only
- `email`: contains `@` and a domain (use a simple regex or library)
- `title` (task): non-empty, max 200 chars

Idea: keep validation **close to the boundary** (in the route handler / controller) so the rest of your code can assume data is clean.

---

# SQL Injection — The Problem

From Module 7 you know how to build SQL queries. Now we look at a **classic attack**: SQL injection.

**Dangerous pattern**:

```
def find_user_by_name(conn, username):
    # ❌ vulnerable: user can inject SQL
    sql = f"SELECT id, username FROM users WHERE username = '{username}'"
    cur = conn.cursor()
    cur.execute(sql)
    return cur.fetchall()
```

If `username` is:

```
alice' OR 1=1 --
```

the resulting SQL becomes:

```
SELECT id, username FROM users WHERE username = 'alice' OR 1=1 --'
```

This may return **all users** instead of just one.

---

# SQL Injection — Safe Pattern with Parameters

Use **parameterized queries** (you started doing this in Module 7).

```python
import sqlite3

def find_user_by_name(conn, username):
    sql = "SELECT id, username FROM users WHERE username = ?"
    cur = conn.cursor()
    cur.execute(sql, (username,))
    return cur.fetchall()

conn = sqlite3.connect(":memory:")
print("This is a demo of parameterized queries, not full setup.")
```

snippet +exec is disabled, run with -x to enable

Key rules:

- **Never** build SQL with `+` / f-strings for untrusted input
- Always use `?` placeholders (`%s` in some drivers) and pass **parameters separately**
- Let the database driver escape dangerous characters safely

This same pattern applies to INSERT, UPDATE, DELETE.

---

# Validating Before Hitting the Database

Combine **validation** + **parameters** for defense in depth.

Example: new student for the Student Manager (Module 9 mini project):

```python
def validate_student_payload(data):
    name = data.get("name", "").strip()
    email = data.get("email", "").strip()
    course = data.get("course", "").strip()

    if not (3 <= len(name) <= 60):
        raise ValueError("Name must be 3–60 characters.")
    if "@" not in email or "." not in email:
        raise ValueError("Invalid email format.")
    if not course:
        raise ValueError("Course is required.")

    return name, email, course
```

Then in your API endpoint or CLI:

1. Call `validate_student_payload()`
2. Use a **parameterized INSERT** into SQLite
3. Catch `ValueError` and return a friendly message / status code

---

# Logging — Helpful vs Dangerous

Logging is useful for debugging and incident response, but it’s easy to leak secrets.

Common mistakes:

- Logging raw passwords or tokens
- Logging entire HTTP bodies with credentials
- Logging full database rows with sensitive columns (e.g., grades, hashes)

Guidelines:

- **Never log passwords** (even hashes are rarely needed in logs)
- Don’t log API keys, JWTs, or session cookies
- Redact or summarize sensitive fields

Bad:

```
print("Login request:", email, password)
```

Better:

```
print("Login attempt for:", email)
```

---

# Storing Passwords — Concepts

You should almost **never** store plain-text passwords.

Concepts (high level, without full crypto details):

- Use a **hash function designed for passwords** (bcrypt, Argon2, PBKDF2)
- Store only the **salted hash**, never the raw password
- On login, hash the provided password and compare to stored hash

In this course:

- We don't implement full auth, but you should know:
  - `hashlib.sha256` is **not ideal** for real passwords (too fast)
  - Libraries like `passlib` or frameworks handle this better

Takeaway: if you ever see code writing password strings directly to a database or CSV, that's a **red flag**.

---

# Config & Secrets Management (Concept)

Where do API keys, DB URLs, and secrets live?

Better options than hard-coding:

- Environment variables (e.g., os.environ["API_KEY"])
- .env files loaded via helper libraries (in real projects)
- Separate config files with **restricted permissions**

Bad:

```
API_KEY = "super-secret-key-123"
```

Better (conceptually):

```
import os
API_KEY = os.environ.get("API_KEY")
```

For this course: avoid committing real secrets to code, and keep demo values clearly fake.

---

# Error Messages & Information Leakage

Error messages help users, but can help attackers too.

Examples:

- Revealing exact SQL queries or stack traces in production
- Returning “User exists, but password is wrong” vs a generic message

Guidelines:

- Show **friendly, minimal** messages to end users
- Log detailed technical info to files (with **no secrets**) for developers
- Avoid exposing framework versions, internal paths, or SQL details in HTTP responses

In your CLI/web mini projects, aim for:

> “Something went wrong, please try again or contact support.”
>
> plus a more detailed log entry for yourself.

---

# Thanks!

By ElnurBDa

# CLI Tools, Logging & Advanced Python

Python Course · Module 11

ElnurBDa

# Module 11 — Command-Line Tools & Advanced Python

In this module we:

- Build **real command-line interfaces (CLIs)** using argparse
- Replace input()-only scripts with tools that accept flags and subcommands
- Introduce **logging** instead of print() for serious scripts
- Explore a few **advanced Python features** you will see in real projects

Focus: make your scripts feel like **professional tools** and understand patterns you'll meet in open-source and production code.

---

# From Scripts to Tools

So far many examples used:

```python
name = input("Enter your name: ")
print("Hello", name)
```

This is great for learning, but less ideal when you want to:

- Automate tasks from other scripts or CI
- Run jobs from cron / Task Scheduler
- Reuse the same script with different options

Command-line tools use **arguments** and **flags** instead:

```python
python greet.py --name Alice
```

We'll use argparse from the standard library to build these.

---

# Basics of argparse

Pattern:

1. Create a parser
2. Define arguments
3. Parse `sys.argv` into a friendly object

```python
import argparse

def main():
    parser = argparse.ArgumentParser(
        description="Greet someone from the command line."
    )
    parser.add_argument("--name", "-n", required=True, help="Name to greet")

    args = parser.parse_args()

    print(f"Hello, {args.name}!")

if __name__ == "__main__":
    main()
```

snippet +exec is disabled, run with -x to enable

Usage:

```python
python greet.py --name Alice
python greet.py -n Bob
```

Try running with `--help` to see automatically generated help text.

---

# Flags, Options & Positional Arguments

argparse supports:

- **Positional args** — required, order matters
- **Options / flags** — usually start with -- or -

Example:

```python
import argparse

def main():
    parser = argparse.ArgumentParser(description="Convert kilometers to miles.")
    parser.add_argument("kilometers", type=float, help="Distance in km")
    parser.add_argument(
        "--precision",
        "-p",
        type=int,
        default=2,
        help="Number of decimal places (default: 2)",
    )

    args = parser.parse_args()
    miles = args.kilometers * 0.621371
    print(f"{args.kilometers} km = {miles:.{args.precision}f} miles")

if __name__ == "__main__":
    main()
```

snippet +exec is disabled, run with -x to enable

Usage:

```python
python convert.py 10
python convert.py 10 --precision 4
```

# Subcommands (Like git commit, git push)

Many tools have **subcommands**:

- `git status`, `git commit`, `git push`
- `docker run`, `docker ps`

We can model this with **sub-parsers**.

```python
import argparse

def cmd_add(args):
    print("Adding task:", args.title)

def cmd_list(args):
    print("Listing tasks; completed =", args.completed)

def build_parser():
    parser = argparse.ArgumentParser(prog="tasks", description="Simple task CLI")
    subparsers = parser.add_subparsers(dest="command", required=True)

    add_p = subparsers.add_parser("add", help="Add a new task")
    add_p.add_argument("title", help="Title of the task")
    add_p.set_defaults(func=cmd_add)

    list_p = subparsers.add_parser("list", help="List tasks")
    list_p.add_argument(
        "--completed",
        action="store_true",
        help="Show only completed tasks",
    )
    list_p.set_defaults(func=cmd_list)

    return parser

def main():
    parser = build_parser()
    args = parser.parse_args()
    args.func(args)  # dispatch to the right handler

if __name__ == "__main__":
    main()
```

snippet +exec is disabled, run with -x to enable

Usage:

```python
python tasks.py add "Study Module 11"
python tasks.py list
python tasks.py list --completed
```

# Connecting CLIs to Files & Databases

You can wire argparse into your existing **file** and **SQLite** code:

- tasks.py add → insert into SQLite (Module 7)
- tasks.py export --format csv → read rows and write CSV (Module 6)
- students.py deactivate --email student@example.com → update database row

Pattern:

- Each subcommand calls into a **function** in another module (db.py, services.py, etc.).
- CLI layer **parses arguments**, then passes them to core logic.

This keeps your business logic **testable** and your CLI thin.

---

# Mini Task — Log Tracker CLI

Extend your **log tracker** project (Module 6) into a CLI:

- Create log_cli.py with argparse:
  - Subcommands: add, list, clear
  - Flags: --user, --action, maybe --level (INFO/WARN/ERROR)
- Internally, call functions from a separate module (e.g., log_core.py) that:
  - open the log file
  - append or read lines
  - handle errors gracefully

Goal: run commands like:

```
python log_cli.py add --user alice --action "login"
python log_cli.py list
python log_cli.py clear
```

---

# Why Logging Beats print()

`print()` is fine while learning, but real programs use **logging**:

- Different **levels** (DEBUG, INFO, WARNING, ERROR, CRITICAL)
- Can write to **files**, not just the console
- Easier to turn on/off detailed output without changing code

Python's `logging` module is built-in and flexible.

---

# Logging Basics

Simple pattern:

```python
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
)

logging.debug("This is a debug message")
logging.info("Starting application")
logging.warning("Low disk space")
logging.error("Something went wrong")
```

snippet +exec is disabled, run with -x to enable

Notes:

- `basicConfig` sets a **global** configuration for the root logger
- By default, output goes to **stderr**
- Change `level=` to control how verbose logs are

Try changing `level=logging.DEBUG` to see all messages.

---

# Log Levels in Practice

Common use:

- DEBUG — very detailed, for developers only
- INFO — high-level events (startup, shutdown, key actions)
- WARNING — something unexpected, but program continues
- ERROR — serious problem, part of the feature failed
- CRITICAL — very bad, program may exit

Example:

```python
import logging

logging.basicConfig(level=logging.DEBUG)

def divide(a, b):
    logging.debug("divide called with a=%s, b=%s", a, b)
    if b == 0:
        logging.error("Attempted division by zero!")
        return None
    return a / b

result = divide(10, 0)
logging.info("Result is %s", result)
```

snippet +exec is disabled, run with -x to enable

Note the %s style placeholders — logging handles formatting lazily.

---

# Logging to a File

Instead of writing only to the console, log to a **file**:

```python
import logging

logging.basicConfig(
    filename="app.log",
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
)

logger = logging.getLogger("demo")

logger.info("Application started")
logger.warning("This is a warning")
logger.error("This is an error")
```

snippet +exec is disabled, run with -x to enable

Open `app.log` in your editor to inspect messages.

Tips:

- Use `getLogger(__name__)` in real modules for better names
- Combine with security best practices (Module 10): **do not** log secrets

---

# Mini Task — Add Logging to an Existing Project

Pick one earlier project:

- File log tracker (Module 6)
- SQLite score tracker (Module 7)
- Student Manager / To-Do backend (Module 9)

Add logging:

1. Configure `logging` in the main entry point.
2. Replace key `print()` calls with appropriate log levels.
3. Ensure that **no sensitive info** (passwords, tokens) is logged.
4. Trigger some errors to see how they appear in logs.

---

# Advanced Python — Comprehensions Recap

You’ve seen list comprehensions. Quick recap:

```
nums = [1, 2, 3, 4, 5]
squares = [n * n for n in nums]
evens = [n for n in nums if n % 2 == 0]

print("squares:", squares)
print("evens:", evens)
```

snippet +exec is disabled, run with -x to enable

Two more related tools:

- **Dict comprehensions**
- **Generator expressions**

We’ll see both next.

---

# Dict Comprehensions

Build dictionaries in one expression:

```
users = ["alice", "bob", "charlie"]

lengths = {name: len(name) for name in users}
print(lengths)
```

snippet +exec is disabled, run with -x to enable

You can also transform existing dicts:

```
scores = {"alice": 90, "bob": 75, "charlie": 82}

passed = {name: s for name, s in scores.items() if s >= 80}
print(passed)
```

snippet +exec is disabled, run with -x to enable

Use case: quick mappings, filtered views, basic data transformations.

---

# Generator Expressions

List comprehensions materialize a list in memory. **Generator expressions** produce values **on demand**:

```
nums = range(1, 1_000_001)

total = sum(n * n for n in nums)
print("Sum of squares from 1 to 1,000,000:", total)
```

snippet +exec is disabled, run with -x to enable

Key differences:

- `[...]` → list comprehension (immediate list)
- `(...)` → generator expression (lazy)

Use generators when:

- The result is large
- You only need to **iterate once** or feed another function like `sum`, `max`

---

# *args and **kwargs

Sometimes you don't know in advance how many arguments a function needs.

- *args — extra **positional** arguments (tuple)
- **kwargs — extra **keyword** arguments (dict)

```
def debug_print(*args, **kwargs):
    print("ARGS:", args)
    print("KWARGS:", kwargs)

debug_print(1, 2, 3, name="alice", active=True)
```

snippet +exec is disabled, run with -x to enable

Typical uses:

- Wrapper functions that pass arguments through
- Flexible APIs where some options are optional/rare

---

# Default Parameters & Keyword-Only Arguments

You’ve seen simple defaults:

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Alice")
greet("Bob", greeting="Hi")
```

snippet +exec is disabled, run with -x to enable

We can also force arguments to be **keyword-only** (for clarity):

```
def create_user(username, *, is_admin=False):
    print("Creating user:", username, "is_admin:", is_admin)

create_user("alice")
create_user("bob", is_admin=True)
# create_user("charlie", True)  # TypeError: must use keyword
```

snippet +exec is disabled, run with -x to enable

Use keyword-only args when positional use would be confusing.

---

# Simple Decorators (Concept)

Decorators are a powerful pattern for **wrapping** functions with extra behavior.

Example: log when a function is called.

```
import time

def log_calls(func):
    def wrapper(*args, **kwargs):
        print(f"[log_calls] calling {func.__name__} with args={args}, kwargs={kwargs}")
        result = func(*args, **kwargs)
        print(f"[log_calls] {func.__name__} returned {result}")
        return result
    return wrapper

@log_calls
def add(a, b):
    return a + b

add(2, 3)
```

snippet +exec is disabled, run with -x to enable

Pattern:

1. Decorator takes a function (`func`)
2. Returns a new function (`wrapper`) that adds behavior
3. `@decorator_name` syntax applies it to `add`

---

# Timing Functions with a Decorator

Another common use: measure how long a function takes.

```python
import time

def timed(func):
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        duration = (time.perf_counter() - start) * 1000
        print(f"{func.__name__} took {duration:.2f} ms")
        return result
    return wrapper

@timed
def slow_operation():
    time.sleep(0.2)

slow_operation()
```

snippet +exec is disabled, run with -x to enable

Real projects use decorators for:

- Logging
- Caching
- Authorization checks
- Retrying failed operations

---

# Mini Project — CLI Admin Tool

Tie everything together by building a **CLI admin tool** for one of your apps:

- Example: Student Manager (students table in SQLite)
- Or: To-Do app tasks (Module 9)

Requirements:

1. Use argparse with subcommands like list, create, deactivate, stats.
2. Connect subcommands to database functions (reuse Module 7 patterns).
3. Use logging to record actions (who, what, when) to a file.
4. Use comprehensions / generator expressions for simple summaries (e.g., count active students).
5. Add at least one decorator (@timed or @log_calls) around a database operation.

You now have the pieces to turn small scripts into **serious tools** that are debuggable, observable, and maintainable.

---

# Thanks!

By ElnurBDa